ASSIGNMENT 1 – CO1

Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Table 1: Submission details

STUDENT NAME	SIRANJEEVI G
REGISTER NUMBER	192572166
COURSE CODE	CSA1720
COURSE NAME	ARTIFICIAL INTELLIGENCE
TOTAL MARKS	50
NATURE OF ASSIGNMENT	INDIVIDUAL
DATE OF SUBMISSION	06.11.2026

Summary of the solution

This report designs, analyses, implements and validates three uninformed search strategies for a robot that carries medicines from the central pharmacy to a ward in a hospital whose state space grows from 1,000 to more than 1,000,000 states. The strategies are **breadth-first search (BFS)** (iterative, queue), **uniform-cost search (UCS)** (iterative, priority queue) and **iterative-deepening depth-first search (IDDFS)** (recursive depth-limited search repeated with increasing limits). Tasks 1–7 give the agent definition, algorithms, operation counts, mathematical complexity, case analysis, asymptotics and the recurrence for the recursive approach. Tasks 8–10 validate the theory with Python experiments on generated hospital graphs of 1,003 / 10,003 / 100,003 / 1,000,003 states. Task 11 recommends the final design and Task 12 re-designs it for the Day-4 localisation failure by searching over belief states. All source code is in Appendix A and raw outputs are in Appendix B.

TASK 1 – Design the Agent and the Search Algorithms

1.1 PEAS description of the robot

Table 2: PEAS description

Component	Specification
Performance measure	Deliver the correct medicine to the requested ward; minimum travel time (path cost); no collisions or unsafe moves; delivery within the time window; low computation time and memory for planning as the hospital grows.
Environment	Multi-block hospital modelled as a graph of locations: junctions, corridors, lifts and ward doors, with staff, patients, trolleys and other obstacles.
Actuators	Drive wheels and steering, lift-call interface, door opener, medicine-drawer release, docking mechanism, alarm or display.
Sensors	Obstacle sensors (LiDAR or ultrasonic), wheel encoders, localisation sensor (beacons or QR/RFID tags), lift and door status, battery gauge, stored hospital map.

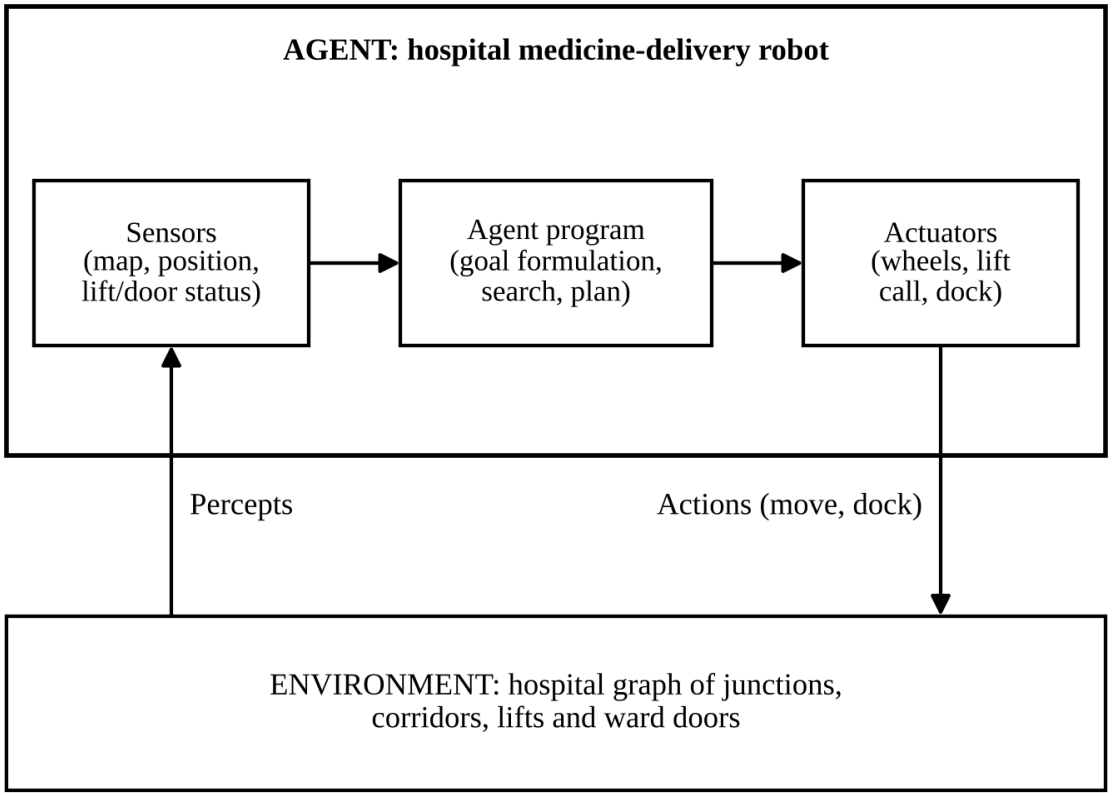


Figure 1: Agent–environment structure of the hospital robot (PEAS view)

1.2 Classification of the task environment

Table 3: Environment properties

Property	Classification	Justification
Observable	Fully observable (planning model, Tasks 1–11)	The map and the robot's own location are known from the localisation sensor, so the state is completely known. This is relaxed in Task 12.
Deterministic	Deterministic	Moving along an edge reliably reaches the connected location at a known cost; lift waiting time is absorbed into the edge cost.
Episodic / sequential	Sequential	A route is a sequence of moves; each move changes the position and so affects later moves and the accumulated cost.
Static / dynamic	Static during planning	The map does not change while the route is computed. Moving people are handled by a low-level reactive layer and by re-planning.

Property	Classification	Justification
Discrete / continuous	Discrete	Finite set of locations, finite actions (move to a neighbour), integer travel-time costs.
Single / multi-agent	Single agent	Only the robot plans; staff and other robots are treated as obstacles whose effect is captured in the edge costs.

1.3 View of AI and rationality

The robot follows the **acting rationally** view of AI: it is a rational agent that, for each percept sequence, selects the action expected to maximise its performance measure. It is a goal-based problem-solving agent because it (i) formulates the goal (be at ward g), (ii) formulates the problem (states, actions, costs), (iii) searches for a sequence of actions that reaches the goal, and (iv) executes that sequence. It is rational because the chosen route is complete (a route is found whenever one exists) and, for BFS and UCS, optimal with respect to hops or travel time respectively, so no other action sequence gives a better value of the performance measure with the same knowledge.

The other views are less suitable. Acting humanly (Turing test) and thinking humanly (cognitive modelling) aim to imitate people, which is irrelevant to delivering medicine. Thinking rationally (laws of thought) needs certain, logically stated knowledge and gives no direct way to choose actions under cost. The rational-agent view is the most general and is measured by behaviour, which suits a delivery robot.

Link to the history of AI. Route finding is a classic state-space search problem. The Dartmouth workshop (1956), the Logic Theorist (Newell, Shaw and Simon, 1956) and the General Problem Solver (1957) established problem solving as search through a space of states. Shortest-path methods such as Moore's breadth-first maze search (1959) and Dijkstra's algorithm (1959, the basis of UCS) came from the same period, and the Shakey robot (1966–72) combined planning with graph search and led to A* (1968). The hospital robot is a modern version of these early problem-solving agents.

1.4 Problem formulation

Table 4: Formal problem definition

Element	Definition
States	Each location of the hospital graph G = (V, E): junction, corridor end, lift stop or ward door. |V| = n takes the values 10^3, 10^4, 10^5, 10^6.
Initial state	The pharmacy node s (node 0).
Actions	ACTIONS(u) = {move to v : (u, v) is an edge of the map}, obtained from the adjacency list of u.
Transition model	RESULT(u, move to v) = v (deterministic).
Goal test	u = g, where g is the requested ward.
Path cost	Sum of edge travel times c(u, v) in seconds, each at least ε = 1 s. The number of moves (depth) is the unit-cost special case.

Symbols used throughout: b = branching factor, d = depth of the shallowest goal, m = maximum depth, C* = optimal path cost, ε = minimum step cost, V = |V|, E = |E|. For the generated hospitals the average degree is 2E/V ≈ 3.8, so b ≈ 3 successors per node after excluding the parent.

1.5 Proposed uninformed search strategies

Three meaningfully different strategies were chosen. They differ in the order of expansion, the data structure that controls it, the objective that is guaranteed and the resource that limits them: BFS is shallowest-first (queue, optimal in hops, memory-hungry), UCS is cheapest-first (priority queue, optimal in travel time) and IDDFS is depth-first with deepening (recursion, tiny memory, repeated work). Plain recursive DFS was not chosen separately because it is the same strategy as the depth-limited search inside IDDFS, and it is neither complete on graphs with loops nor optimal.

Table 5: Summary of the selected strategies

Strategy	Style	Basic idea	Why selected
BFS	Iterative, FIFO queue	Expand all nodes at depth k before depth k+1.	Fewest moves; fast baseline; shows memory growth.

Strategy	Style	Basic idea	Why selected
UCS	Iterative, priority queue	Always expand the lowest path-cost node.	Lowest travel time, which is the real objective.
IDDFS	Recursive, depth-limited DFS repeated	Run DLS with limit 0, 1, 2, … until the goal is found.	Memory O(d); shows time-versus-memory trade-off.

1.5.1 Preprocessing required

- **Adjacency lists** (all algorithms): the map database gives an edge list. One pass creates adj[u] and the cost list for both directions, time $\Theta(V + E)$ and space $\Theta(V + E)$. It is done once and re-done only when a wing is added.
- **Explored set / parent table** (BFS and UCS): a hash table keyed by state that gives O(1) expected duplicate detection and stores the parent for path reconstruction. UCS also keeps dist[] and a closed set.
- **Current-path set** (IDDFS): only the nodes on the current path are stored (O(m)); no global explored set is kept, which is what makes its memory small.
- **Optional connected-component labelling** (one BFS flood fill, $\Theta(V + E)$): lets an unreachable ward be rejected in O(1). It is used in the final design (Task 11).

1.5.2 Breadth-first search – idea, pseudocode and flowchart

Idea. A FIFO queue holds the frontier, so nodes leave in order of depth and the first time the goal is generated it is at the shallowest depth. The explored set prevents re-generation of states reached through loops. **Justification.** It is complete and gives the route with the fewest moves, and it is the natural baseline for memory growth.

```
1. function BFS(adj, s, g):
    2. if s = g then return [s]
    3. parent[s] ← NIL;  Q ← queue containing s
    4. while Q is not empty do
        5. u ← DEQUEUE(Q)                          // node expansion
        6. for each v in adj[u] do                 // successor generation
            7. if v is not a key of parent then    // duplicate detection
                8. parent[v] ← u
                9. if v = g then return PATH(parent, g)   // goal test at generation
```

10. ENQUEUE(Q, v)
11. return FAILURE

Flowchart of BFS

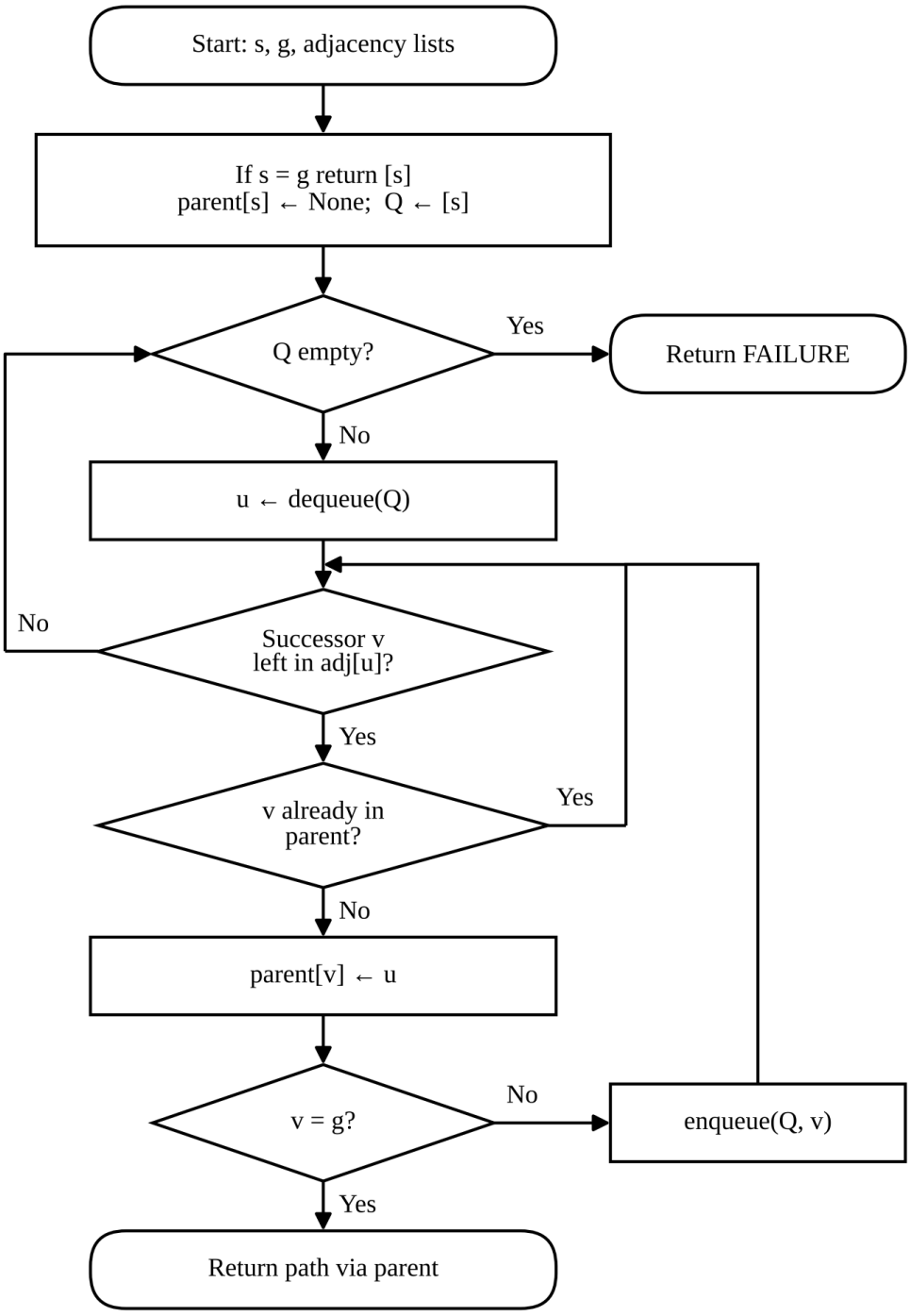


Figure 2: Flowchart of breadth-first search

1.5.3 Uniform-cost search – idea, pseudocode and flowchart

Idea. A priority queue ordered by path cost g(n) always releases the cheapest frontier node. The goal test is applied when a node is expanded (not when it is generated) because a cheaper route to it may still be found. A stale entry is skipped using the closed set. **Justification.** Edges

have different travel times, so the minimum-hop route of BFS is not the minimum-time route; UCS is complete and optimal when every cost is at least $\varepsilon > 0$.

```
1.  function UCS(adj, c, s, g):
2.      dist[s] ← 0;  parent[s] ← NIL;  PQ ← {(0, s)};  closed ← empty
3.      while PQ is not empty do
4.          (d, u) ← POP-MIN(PQ)                    // node expansion
5.          if u in closed then continue            // stale entry
6.          if u = g then return PATH(parent, g), d      // goal test at expansion
7.          add u to closed
8.          for each (v, w) in adj[u] do            // successor generation
9.              if v not in closed and d + w < dist[v] then  // dist default = infinity
10.                 dist[v] ← d + w;  parent[v] ← u;  PUSH(PQ, (d + w, v))
11.     return FAILURE
```

Flowchart of UCS

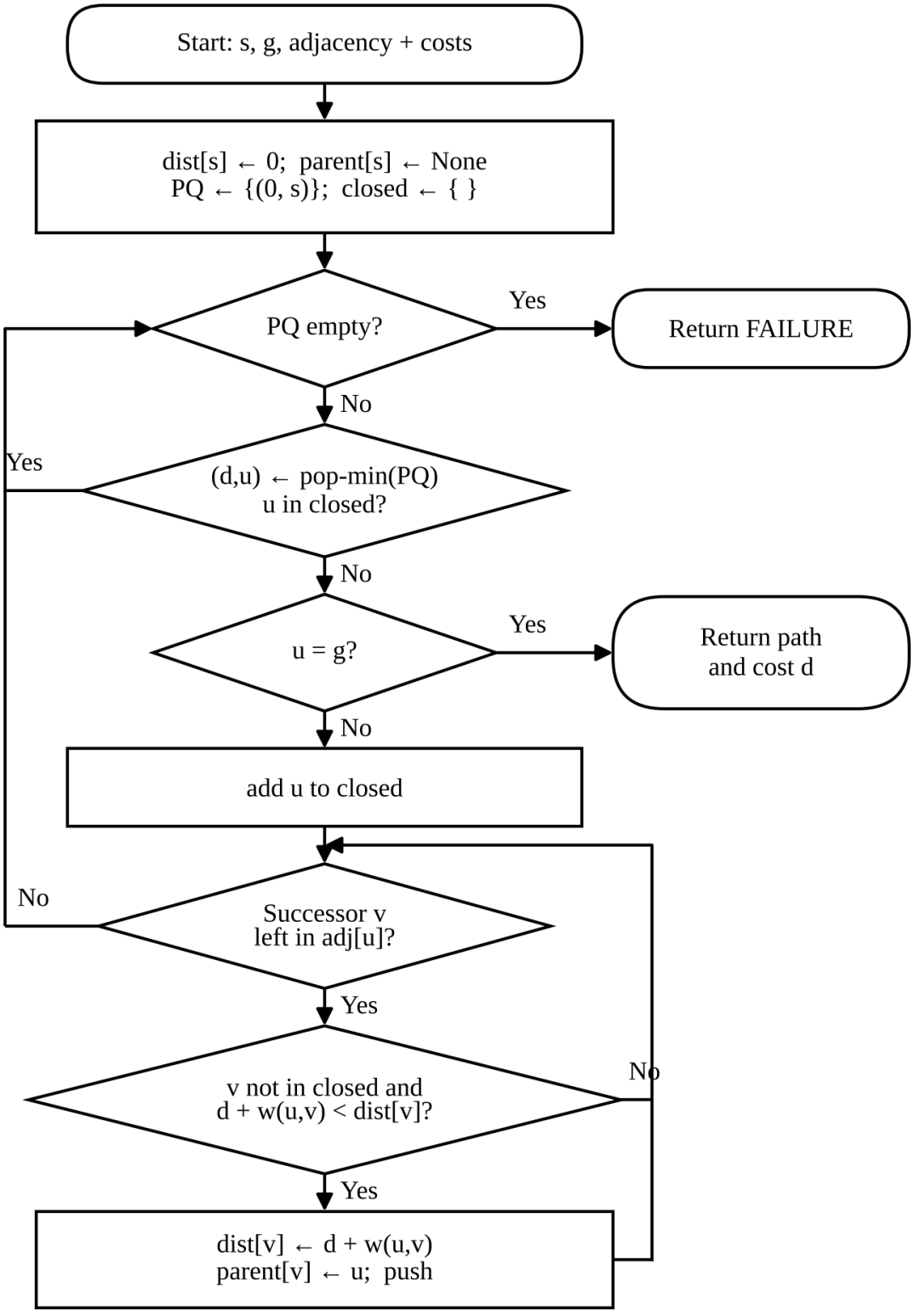


Figure 3: Flowchart of uniform-cost search

1.5.4 Iterative-deepening DFS (recursive) – idea, pseudocode and flowcharts

Idea. Depth-limited search (DLS) is a recursive depth-first search that stops at a depth limit. IDDFS calls DLS with limit 0, 1, 2, … so the first solution found is the shallowest, while memory stays proportional to the depth. Only the current path is checked for cycles. **Justification.** It is the recursive approach required by the assignment, and it represents the low-memory end of the trade-off.

```
1. function IDDFS(adj, s, g):
2.     for limit ← 0, 1, 2, … do
3.         r ← DLS(s, limit)
4.         if r = found then return path (recorded while unwinding)
5.         if r = fail then return FAILURE          // nothing was cut off: space exhausted
1. function DLS(u, limit):                          // recursive
2.     if u = g then return found
3.     if limit = 0 then return cutoff
4.     add u to onPath;  cut ← false
5.     for each v in adj[u] with v not in onPath do
6.         r ← DLS(v, limit − 1)                    // recursive call
7.         if r = found then remove u from onPath; return found
8.         if r = cutoff then cut ← true
9.     remove u from onPath
10.    return cutoff if cut else fail
```

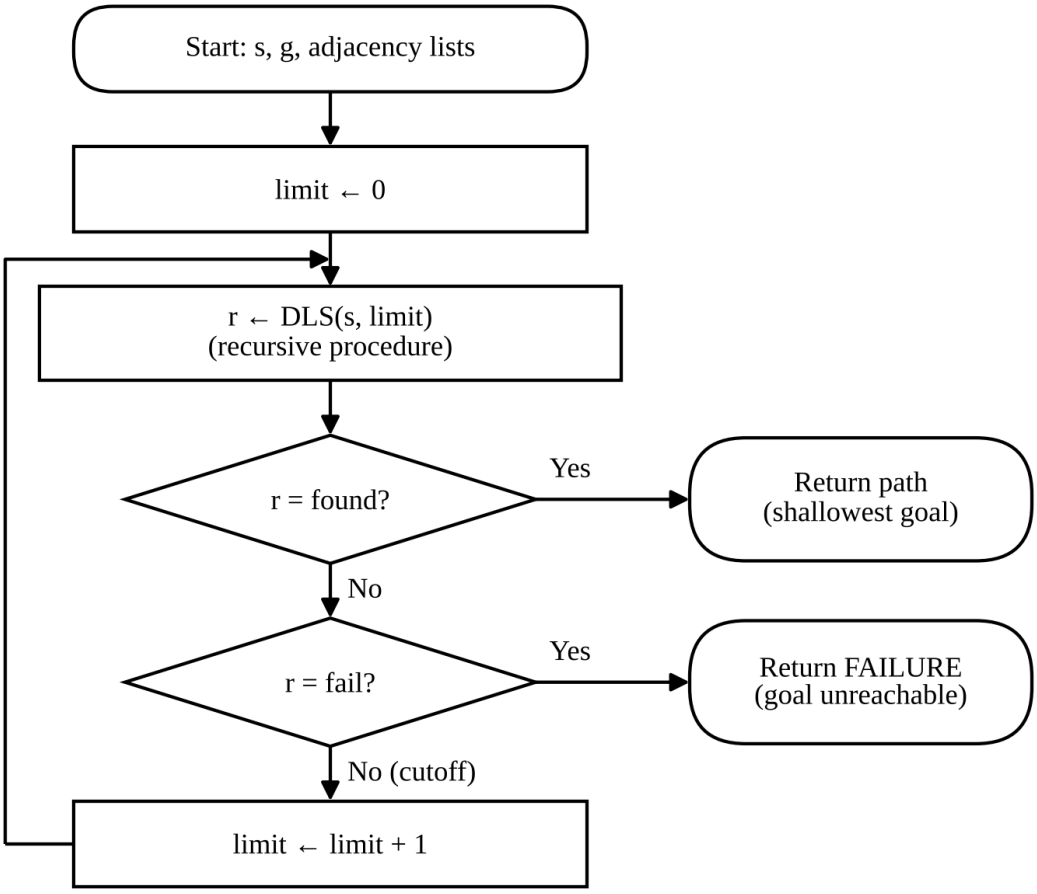


Figure 4: Flowchart of the IDDFS driver loop

Flowchart of the recursive depth-limited search

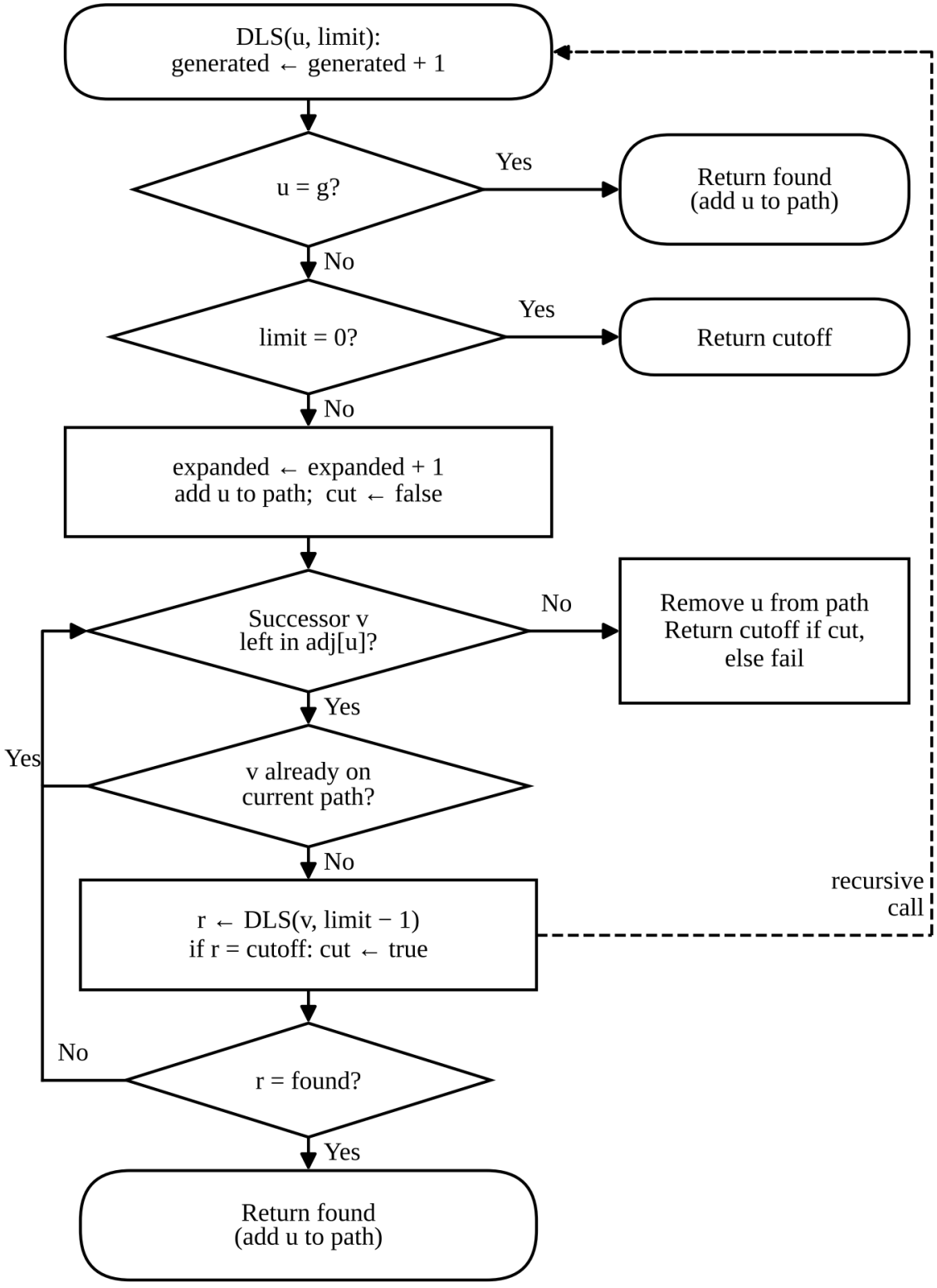


Figure 5: Flowchart of the recursive depth-limited search (DLS); the dashed arrow is the recursive call

1.6 Example hospital used to illustrate the algorithms

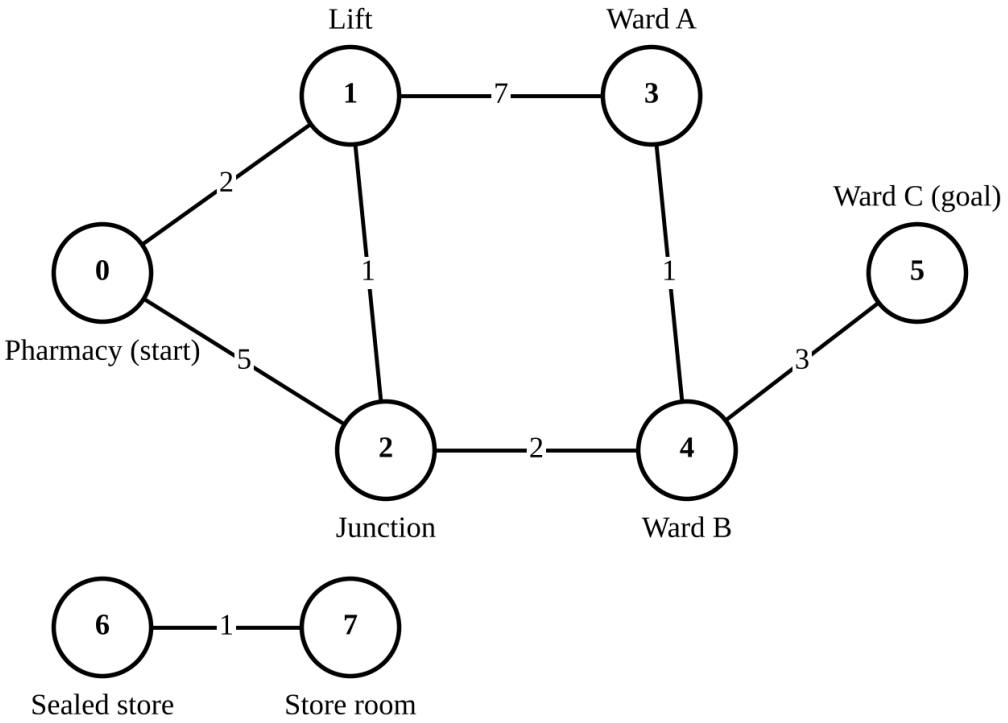


Figure 6: Example hospital graph with travel times on the edges (node 0 = pharmacy, node 5 = Ward C)

Adjacency lists built by the preprocessing step: adj[0] = {1, 2}; adj[1] = {0, 2, 3}; adj[2] = {0, 1, 4}; adj[3] = {1, 4}; adj[4] = {2, 3, 5}; adj[5] = {4}; adj[6] = {7}; adj[7] = {6}. For the request 0 → 5, BFS returns 0-2-4-5 (3 moves, cost 10), UCS returns 0-1-2-4-5 (4 moves, cost 8) and IDDFS returns 0-2-4-5 (limit 3). The example shows that fewest moves is not the same as least time. Nodes 6 and 7 are a sealed store and form an unreachable goal.

TASK 2 – Identify the Basic Operation

The basic operation of a search algorithm is the operation executed most often and proportional to the running time. In all three algorithms it lies in the **successor-generation step**, because node expansion happens once per node but the inner loop runs once per outgoing edge.

Table 6: Basic operations

Algorithm	**Basic operation and where it occurs**	**Why appropriate**	**Growth as the state space grows**
BFS	Successor generation: one iteration of the inner for-loop (line 6): duplicate test, parent insertion, goal test and enqueue.	Executed once per edge leaving an expanded node, so it is at least as frequent as dequeue and all other steps; each execution costs O(1) (hash and deque).	Count ≈ b × expanded nodes. The expanded set grows like b^d until it is bounded by the whole graph, so the count grows linearly with |V| and never exceeds 2E.
UCS	Edge relaxation with frontier insertion: inner loop (lines 8–10) comparing d + w with dist[v] and pushing onto the heap.	Every successor is relaxed once per expansion, so pushes and relaxations are proportional; each push costs O(log n), so it fixes the time.	Relaxations ≤ 2E; grows with the number of nodes cheaper than C*, i.e. up to depth C*/ε, and up to all of V.
IDDFS	Node generation: one call to DLS (line 1 of DLS: goal test, limit test, loop start).	Each recursive call does constant work plus a loop over successors, so the number of calls measures the work; no other step repeats more often.	Calls over all iterations $\Sigma(d - i + 1)\, b^i$: exponential in d, so it multiplies by about b for every extra level.

The experimental counter succ_checks counts exactly these inner-loop executions (Table 7). The counts confirm the discussion: at the 1,003-state hospital BFS performs 678 inner-loop steps, and at 1,000,003 states it performs 659,081, a growth of about 970 times for 1,000 times more states. IDDFS performs 143,399 steps at 10,003 states and 1,921,631 at 100,003 states.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically

3.1 Input size and assumptions

Let b be the branching factor (successors per node, parent excluded), d the depth of the shallowest goal, m the maximum depth, C* the optimal cost, ε the minimum step cost, $V = |V|$ and $E = |E|$. The tree model assumes that level i contains b^i nodes. The graph model uses the explored set, so no state is generated twice and every count is bounded by V and 2E.

3.2 Preprocessing cost (all algorithms)

Creating V empty lists costs c_1V. Each of the E edges is appended to two lists (four appends including the cost lists), costing c_2E. Hence

$$T_{pre}(V, E) = c_1V + c_2E = \Theta(V + E), \quad S_{pre} = \Theta(V + E).$$

With $E \approx 1.9V$ in the hospital model, $T_{pre} = \Theta(V)$. It was measured at 3.97 s for 1,000,003 states (Table 5). The explored set starts empty (O(1)) and each insertion or lookup is O(1) expected.

3.3 Breadth-first search

BFS with goal test at generation generates at most the nodes of levels 1 to d. The number of nodes generated is the geometric sum

$$N_{gen}(BFS) = 1 + \Sigma_{i=1}^{d} b^i = (b^{d+1} - 1) / (b - 1).$$

Nodes expanded are those of levels 0 to $d - 1$: $N_{exp} = \Sigma_{i=0}^{d-1} b^i = (b^d - 1)/(b - 1)$. Each expansion runs the inner loop b times, so the basic operation executes

$$T(BFS) = b \times N_{exp} = b\,(b^d - 1)/(b - 1) = \Theta(b^d).$$

Space: at the moment the goal is generated the queue holds up to the whole of level d, and the explored set holds every generated node, so $S(BFS) = \Theta(b^d)$. In the graph model $N_{gen} \leq V$ and $T \leq 2E$, so $T = O(\min(b^d, V + E))$.

Overall cost. $T_{total} = \Theta(V + E) + O(\min(b^d, V + E))$. For one request the preprocessing dominates; because the map is built once and reused for many deliveries, the amortised cost per request is $O(\min(b^d, V + E))$.

3.4 Uniform-cost search

UCS expands every node whose path cost is below C* (and some with cost C*). Because every step costs at least ε, a node with $g(n) \leq C^*$ lies at depth at most $\lfloor C^*/\varepsilon \rfloor$, so the tree model gives

$$N_{exp}(UCS) \leq \Sigma_{i=0}^{\lfloor C^*/\varepsilon \rfloor + 1} b^i = O(b^{1 + \lfloor C^*/\varepsilon \rfloor}).$$

When $C^*/\varepsilon \gg d$ (cheap-edge detours) this is far larger than BFS. In the graph model each node is expanded at most once and each directed edge is relaxed at most once, so there are at most 2E relaxations and at most 2E + 1 heap entries. A binary-heap push or pop costs O(log H) with $H \leq 2E + 1$, therefore

$$T(UCS) = O((V + E) \log V), \quad S(UCS) = O(V + E) \text{ in the worst case, } \Theta(b^{1 + \lfloor C^*/\varepsilon \rfloor}) \text{ in the tree model.}$$

For the hospital $E = \Theta(V)$, so $T = O(V \log V)$. The overall cost is $T_{total} = \Theta(V + E) + O((V + E) \log V)$, which is dominated by the search term for large requests.

3.5 Iterative-deepening DFS

A depth-limited search with limit l generates $\Sigma_{i=0}^{l} b^i$ nodes (derived formally in Task 7). IDDFS repeats this for $l = 0, \ldots, d$, so level i is generated $(d - i + 1)$ times:

$$N_{gen}(IDDFS) = \Sigma_{l=0}^{d} \Sigma_{i=0}^{l} b^i = \Sigma_{i=0}^{d} (d - i + 1)\, b^i = (b^{d+2} - (d + 2)\, b + (d + 1)) / (b - 1)^2.$$

Check: $d = 0$ gives 1; $d = 1$ gives $b + 2$ (1 call at limit 0, $1 + b$ calls at limit 1). Asymptotically $N_{gen} \approx b^d\, (b/(b - 1))^2 = \Theta(b^d)$. Compared with BFS, which generates about $b^d \cdot b/(b - 1)$, the extra work is only the factor $b/(b - 1)$ (1.5 for $b = 3$). The basic-operation count is $T = b \times N_{exp} = \Theta(b^d)$.

Space: the recursion stack holds at most d + 1 frames. If every frame stores its unexplored siblings the memory is O(b d); with a successor iterator, as in the implementation, it is O(d). The current-path set adds O(d). Because only the current path is checked for duplicates, a graph with loops contains many repeated simple paths, so in the worst case (goal unreachable) the time becomes $O(b^m)$ with m up to $V - 1$. The overall cost is $T_{total} = \Theta(V + E) + O(b^d)$.

Table 7: Summary of derived complexities

Algorithm	**Nodes generated**	**Time (basic operation)**	**Space (frontier)**	**Preprocessing**
BFS	$(b^{d+1} - 1)/(b - 1)$	$\Theta(b^d)$; $O(V + E)$ in a graph	$\Theta(b^d)$	$\Theta(V + E)$
UCS	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O((V + E) \log V)$ in a graph	$\Theta(b^{1+\lfloor C^*/\varepsilon \rfloor})$ / $O(V + E)$	$\Theta(V + E)$
IDDFS	$(b^{d+2} - (d+2)b + d + 1)/(b - 1)^2$	$\Theta(b^d)$; $O(b^m)$ worst case	O(b d) or O(d)	$\Theta(V + E)$

TASK 4 – Analyse Best, Average and Worst-Case Behaviour

The cases depend on where the goal lies and on how the graph is explored, not on the size of the graph alone. Measured node counts (generated nodes) from Tables 1 and 2 support the reasoning; IDDFS entries marked DNF exceeded the budget of 2,000,000 node generations.

4.1 Breadth-first search

- **Best case:** s = g (1 node generated, 0 expanded) or the goal is the first neighbour of s: only s is expanded and the goal is generated immediately, so N = 2 and the time is $\Theta(1)$ ($\Theta(b)$ if the goal is the last neighbour). Measured: 2 generated, 1 expanded.
- **Average case:** the goal is a uniformly random reachable node. Most nodes lie in the last levels, so on average BFS finishes all levels above the goal's level and about half of that level: $N_{avg} \approx V/b + (1/2)(1 - 1/b)V \approx (b + 1)V/(2b)$, which is between V/b and V. Measured: 39% of V (393 of 1,003) at 1,000 states and 43% of V (4,324 of 10,003) at 10,000 states over 20 random goals.
- **Worst case:** the goal is the last node of the deepest level, or is unreachable. BFS must generate every node of the component, N = V, with 2E basic operations, $\Theta(V + E)$. Measured: the unreachable goal generated and expanded all 1,000,000 nodes.

4.2 Uniform-cost search

- **Best case:** s = g, or the goal is joined to s by the cheapest edge out of s: the first pop expands s and the second pop returns the goal (expanded 1). Measured: 3 generated, 1 expanded.
- **Average case:** the goal has a typical cost C; every node with cost below C is expanded, which is roughly the fraction of the graph within cost C. Measured mean generated: 818 of 1,003 states and 8,127 of 10,003 states (about 82% of V), more than BFS because UCS also re-inserts nodes whose cost improves.
- **Worst case:** the goal is expensive, or reachable only through a long chain of cheap edges (cheap-edge detour), or unreachable. All nodes with cost below C* are expanded, up to depth C*/ε, and in the limit the entire graph with heap overhead O(log V). Measured: all 1,000,000 nodes expanded, 1,209,244 generated.

4.3 Iterative-deepening DFS

- **Best case:** s = g, or the goal is at depth 1: iterations 0 and 1 give 1 + 2 = 3 generated nodes. A goal on the first branch at depth $d \geq 2$ is not cheap because iterations 0 to $d - 1$ must complete first: $N \approx \Sigma_{l<d} (b^{l+1} - 1)/(b - 1) = \Theta(b^{d-1})$. Measured best case: 3 generated.
- **Average case:** the final iteration (limit d) is on average half completed, and all earlier iterations are complete: $N_{avg} \approx (1/2)\, b^d\, (b/(b - 1))^2 + O(b^{d-1})$. In a graph with loops each state is reached by many simple paths, so the count exceeds this tree estimate. Measured mean: 3,422 generated at 1,003 states (3.4 times V) and 133,557 at 10,003 states.
- **Worst case:** the goal is on the last branch at depth d (N is the full formula of Task 3), or the goal is unreachable. For an unreachable goal the search stops only when no node is cut off, which needs depth up to the longest simple path, so $N = O(b^m)$ with m up to $V - 1$. Measured: DNF (more than 2,000,000) even at 1,003 states.

Table 8: Nodes generated in the best, average and worst cases, |V| = 1,003

Case	BFS	UCS	IDDFS
Best (goal = first neighbour)	2	3	3
Average (mean of 20 random goals)	393	818	3,422
Worst, deepest node (depth 7)	719	847	8,809
Worst, unreachable goal	1,000	1,228	DNF

Table 9: Nodes generated in the best, average and worst cases, |V| = 10,003

Case	BFS	UCS	IDDFS
Best (goal = first neighbour)	2	3	3
Average (mean of 20 random goals)	4,324	8,127	133,557
Worst, deepest node (depth 9)	6,413	11,961	208,444
Worst, unreachable goal	10,000	12,087	DNF

The tables show the pattern predicted above: the best case is independent of V, the average case is a sizeable fraction of V for BFS and UCS, and the worst case equals V for BFS and UCS while IDDFS never finishes on an unreachable goal.

TASK 5 – Perform Asymptotic Analysis

Table 10: Asymptotic behaviour

Algorithm	Time	Memory	Complete	Optimal
BFS	$\Omega(1)$, $O(b^d) \leq O(V + E)$; $\Theta(V)$ in the hospital	$\Theta(b^d) \leq \Theta(V)$	Yes	Fewest moves only
UCS	$\Omega(1)$, $O((V + E) \log V)$; $\Theta(V \log V)$ worst case	$O(V + E)$ worst, $\Theta(b^{1+\lfloor C^*/\varepsilon \rfloor})$ tree	Yes ($\varepsilon > 0$)	Least travel time
IDDFS	$\Theta(b^d)$; $O(b^m)$ on loops	$\Theta(d)$ or $\Theta(b\ d)$	Yes (finite graph)	Fewest moves only

Large state spaces. In the hospital, depth grows logarithmically with size ($d \approx \log_b V$), so $b^d \approx V$ and BFS time and memory are both $\Theta(V)$. The measured exponent of nodes generated against V is 1.003 for BFS (333 to 339,565 over a 1,000-fold growth). UCS adds the logarithmic heap factor, $\Theta(V \log V)$; its measured exponent (0.80) is below 1 only because the random goal at 1,000,003 states needed 30% of the graph. IDDFS has time $\Theta(b^d)$ in a tree, but with loops the number of simple paths grows faster than V: nodes generated grew 57.6 times for the step from 1,003 to 10,003 states and 13.5 times from 10,003 to 100,003, and the search did not finish at 1,000,003.

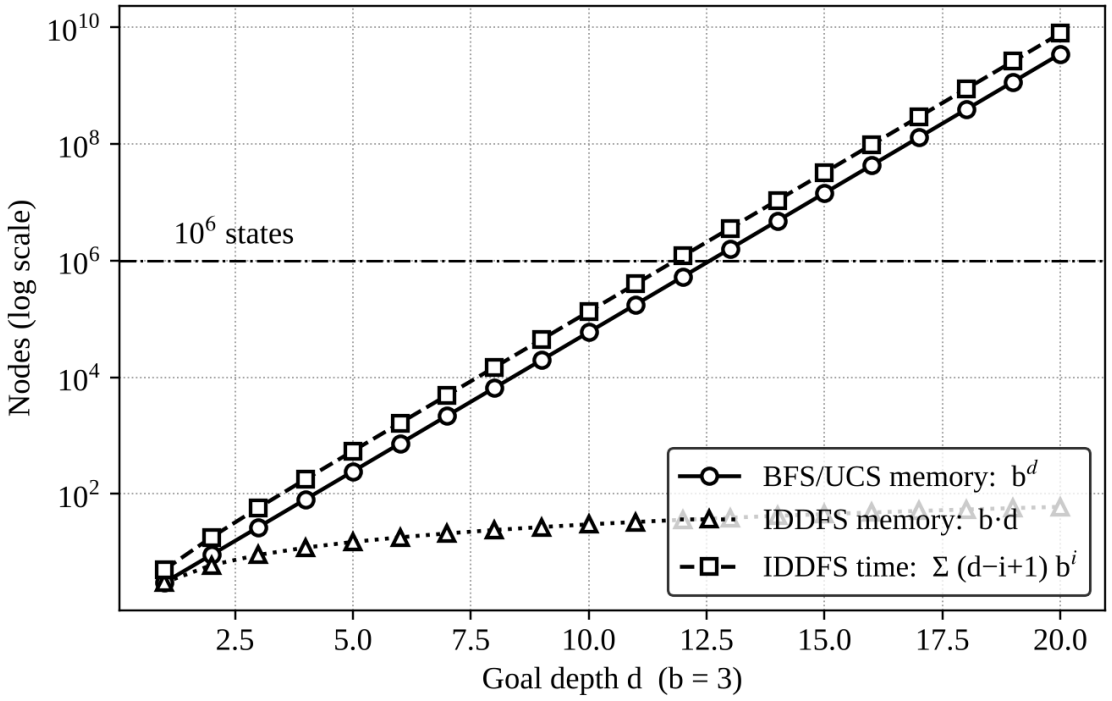


Figure 7: Growth of memory and time with goal depth for b = 3 (log scale)

Time versus memory as the hospital grows. For BFS the frontier holds 0.23 V entries for the reachable goal and 0.66 V for the worst case (658,667 entries at 1,000,003 states). Measured peak memory of the search structures was about 17 MB (BFS) and 37 MB (UCS) for the reachable goal, and 67 MB and 153 MB when the whole 1,000,003-state graph was explored. Search time was only 0.26 s and 1.5 s (BFS) and 0.84 s and 7.6 s (UCS). Memory grows linearly while an onboard computer has a fixed RAM, so **memory becomes the limiting factor first for BFS and UCS** (at about 10^7 states the worst-case tables exceed 1 GB). For IDDFS the memory is only d + 1 entries (13 at 10^6 states), but time explodes first: **time is the limiting factor for IDDFS**, which already exceeds 2,000,000 node generations at 1,000,003 states.

TASK 6 – Formulate the Recurrence Relation

The recursive approach is the depth-limited search DLS(u, limit) inside IDDFS. The recurrence is read directly from the pseudocode of Section 1.5.4. Let T(l) be the number of nodes generated (calls of DLS) by DLS with remaining depth l, in the worst case where the goal is not found and every node is expanded (tree model with b successors per node after excluding the parent).

Table 11: Components of the recurrence

Component	Identification from the DLS algorithm
Size of the original problem	The remaining depth limit l (the call DLS(u, l)).
Size of each subproblem	$l - 1$, because every recursive call is DLS(v, limit − 1).
Number of recursive calls	b: the for-loop in lines 4–5 makes one call for every successor v not on the current path.
Work outside the recursive calls	Constant, 1 node generation per call (goal test, limit test, path-set update). If successor checks are counted instead, the loop adds b per call.
Base case	$l = 0$: the call performs the goal test and returns cutoff without recursion, so $T(0) = 1$.

Combining the components, a call with remaining depth l performs 1 unit of work and then b recursive calls with remaining depth $l - 1$:

$$T(l) = b \cdot T(l - 1) + 1 \quad \text{for } l \geq 1, \quad T(0) = 1.$$

Counting successor checks (the basic operation of Task 2) instead of node generations gives, because the loop runs b times in every expanded call and not at all at $l = 0$:

$$T_s(l) = b \cdot T_s(l - 1) + b \quad \text{for } l \geq 1, \quad T_s(0) = 0.$$

For the space used by the recursion, one new stack frame is added per level, so $S(l) = S(l - 1) + 1$ with $S(0) = 1$. The IDDFS outer loop repeats DLS for $l = 0, \ldots, d$, so its total is $I(d) = \Sigma_{l=0}^{d} T(l)$.

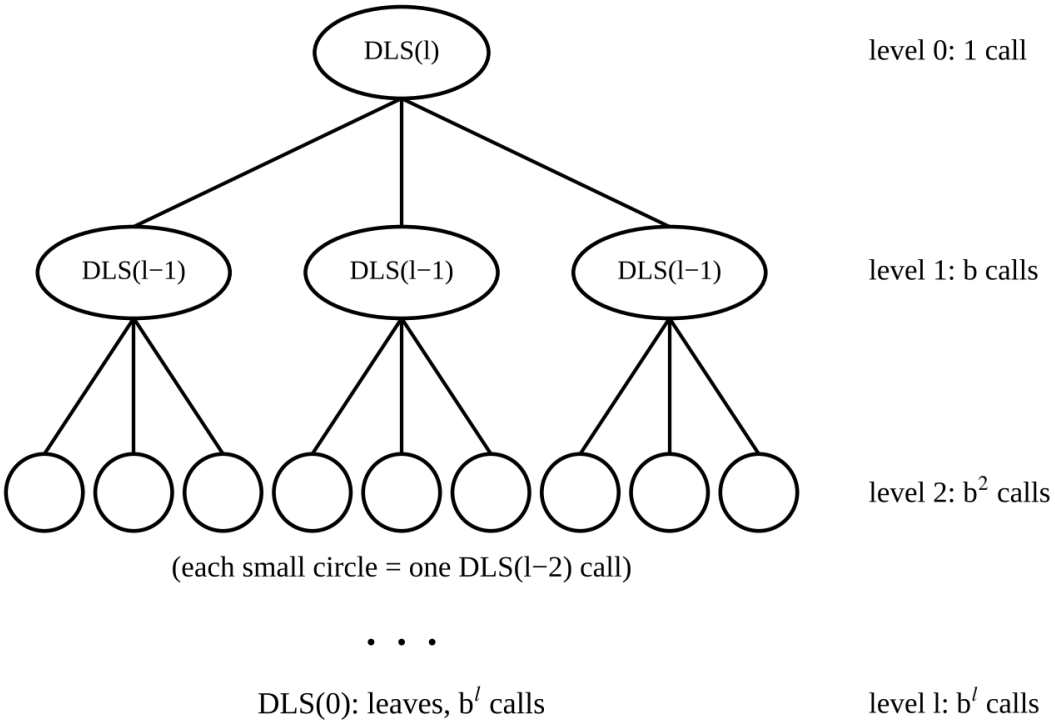


Figure 8: Recursion tree of DLS for b = 3: level i contains b^i calls each doing one unit of work

TASK 7 – Solve the Recurrence Relation

7.1 Choice of method

The recurrence $T(l) = b\,T(l-1) + 1$ reduces the problem size by a constant ($l \to l-1$) instead of dividing it, so the Master theorem for $T(n) = aT(n/c) + f(n)$ does not apply. **Iteration (repeated substitution)** is suitable because the recurrence is linear with a constant coefficient, and the result is verified by the **recursion tree** and by **induction**.

7.2 Solution by iteration

$$T(l) = b\,T(l-1) + 1$$

$$= b\,(b\,T(l-2) + 1) + 1 = b^2\,T(l-2) + b + 1$$

$$= b^3\,T(l-3) + b^2 + b + 1$$

$$= b^k\,T(l-k) + \Sigma_{i=0}^{k-1}\,b^i.$$

The substitution stops when $l - k = 0$, i.e. $k = l$. With $T(0) = 1$:

$$T(l) = b^l + (b^l - 1)/(b-1) = (b^{l+1} - 1)/(b-1) = \Theta(b^l) \quad (b > 1).$$

For $b = 1$ the sum has l terms and $T(l) = l + 1 = \Theta(l)$.

7.3 Verification by recursion tree and induction

In the recursion tree (Figure 9) level i contains b^i calls, each doing one unit of work outside its recursive calls, for levels $i = 0, \ldots, l$. The total work is $\Sigma_{i=0}^{l}\,b^i = (b^{l+1} - 1)/(b-1)$, the same result. Induction: for $l = 0$ the formula gives $(b-1)/(b-1) = 1 = T(0)$. If it holds for $l-1$, then $T(l) = b\,(b^l - 1)/(b-1) + 1 = (b^{l+1} - b + b - 1)/(b-1) = (b^{l+1} - 1)/(b-1)$. ■

For the successor-check form, $T_s(l) + b/(b-1) = b\,[T_s(l-1) + b/(b-1)]$ with $T_s(0) + b/(b-1) = b/(b-1)$, which gives $T_s(l) = b\,(b^l - 1)/(b-1) = b \times$ (number of expanded nodes), consistent with Task 3.

7.4 Interpretation

- The cost of DLS grows exponentially with the depth limit: every extra level multiplies the work by b. For $b = 3$, $T(10) = 88{,}573$, $T(13) = 2{,}391{,}484$ and $T(20) = 5{,}230{,}176{,}601$.
- For fixed depth the cost grows as a polynomial of degree l in b, so a small increase of the branching factor (extra cross-corridors) has a large effect. This is the reason IDDFS failed on the loop-rich 1,000,003-state graph.
- The memory recurrence $S(l) = S(l-1) + 1$ gives $S(l) = l + 1 = \Theta(l)$, linear, which explains the tiny measured memory.

7.5 Extension to iterative deepening

IDDFS calls DLS for $l = 0, \ldots, d$, so its cost is the sum of the solutions:

$$I(d) = \Sigma_{l=0}^{d} (b^{l+1} - 1)/(b - 1) = [\ (b^{d+2} - b)/(b - 1) - (d + 1)\ ] / (b - 1) = (b^{d+2} - (d + 2)\, b + d + 1)/(b - 1)^2.$$

The ratio to the last iteration is $I(d)/T(d) \to b/(b - 1)$: about 1.5 for $b = 3$ and 1.11 for $b = 10$. Repeating the earlier iterations therefore costs only a constant factor, because the last level contains most of the nodes, and IDDFS remains $\Theta(b^d)$ in time and $\Theta(d)$ in space.

TASK 8 – Implement and Test the Algorithms

8.1 Implementation

The algorithms are implemented in Python 3.12 (hospital_search.py, Appendix A) exactly as in the pseudocode of Task 1: bfs() uses collections.deque with a parent dictionary, ucs() uses heapq with dist, parent and closed tables and lazy deletion of stale entries, and iddfs() uses a recursive nested function dls() with a current-path set. Each function returns the path and the counters generated, expanded, max_frontier and succ_checks. For IDDFS a budget of 2,000,000 node generations stops runs that do not finish (reported as DNF).

8.2 Test graphs, seeds and procedure

Hospital graphs are generated by generate_edges(): a backbone corridor tree with fan-out about 3 (pharmacy = node 0), one random cross-link per node (loops between nearby locations on a level), integer travel times uniform in 1–9 s, and a sealed island of three locations that is used as the unreachable goal. **Random seeds:** graph seed 2026 and goal-selection seed 7. The same generator, seeds, start (node 0) and goals are used for all three algorithms, so the comparison is on identical inputs. Each timing is the mean of 5 runs.

Table 12: Test inputs and preprocessing cost (adjacency-list construction)

States |V|	Edges |E|	Avg degree	Goal node	Goal depth	Max depth	Preproc. (ms)
1,003	1,919	3.83	390	6	7	0.7
10,003	19,043	3.81	6355	8	9	4.9
100,003	190,788	3.82	46631	10	11	207.0
1,000,003	1,906,360	3.81	329752	12	13	4342.5

Reproducible procedure: (1) python3 test_correctness.py for the correctness tests, (2) python3 run_experiments.py for all sizes and scenarios (writes results.json), (3) python3 belief_states.py for Task 12. Test cases per size: reachable goal, unreachable goal (node |V| − 1) and start equal to goal.

8.3 Correctness tests and expected behaviour

Table 13: Expected behaviour and observed outcome

Test case	Expected behaviour	Observed (all sizes)
Reachable goal	A valid route; BFS and IDDFS give the minimum number of moves; UCS gives the minimum travel time.	All routes validated edge by edge; BFS and UCS correct at every size; IDDFS correct up to 100,003 states, DNF at 1,000,003.
Unreachable goal	BFS and UCS return FAILURE after exploring the component; IDDFS should terminate with FAILURE.	BFS and UCS return FAILURE. IDDFS did not finish within the budget at any size (it terminates only on tiny graphs, Test 1).
Start = goal	Path [s], cost 0, no expansion.	All three return [s], cost 0, 1 generated, 0 expanded.

Output of test_correctness.py (Appendix B): on the tiny hospital of Figure 7 all three algorithms behave as expected, and on 300 random small graphs (2,554 start/goal pairs) compared against Bellman–Ford distances and BFS hop counts there were **0 failures**. The implementation therefore corresponds to the algorithms analysed in Tasks 1–7.

```
TEST 1: tiny hospital  (expected: BFS/IDDFS fewest hops, UCS least time)
    reachable Ward C   BFS   path=[0, 2, 4, 5] cost=10 status=ok
    reachable Ward C   UCS   path=[0, 1, 2, 4, 5] cost=8 status=ok
    reachable Ward C   IDDFS path=[0, 2, 4, 5] cost=10 status=ok
    start = goal       BFS   path=[0] cost=0 status=ok
    start = goal       UCS   path=[0] cost=0 status=ok
    start = goal       IDDFS path=[0] cost=0 status=ok
    unreachable store  BFS   path=None cost=None status=nosol
    unreachable store  UCS   path=None cost=None status=nosol
    unreachable store  IDDFS path=None cost=None status=nosol
TEST 2: 2554 start/goal pairs on 300 random graphs -> failures: 0
```

TASK 9 – Record Experimental Performance

The data below were produced by run_experiments.py (Appendix A; log in Appendix B). Nodes generated are the nodes created after duplicate detection (UCS counts every insertion); nodes expanded are those whose successors were generated; the maximum frontier is the largest number of states waiting in the queue (BFS, UCS) or the maximum recursion depth (IDDFS). Times are means of 5 runs; DNF means the budget of 2,000,000 node generations was exceeded.

Table 14: Reachable goal: search effort

Algorithm	|V|	Generated	Expanded	Max frontier	Time (ms)
BFS	1,003	333	112	221	0.219
UCS	1,003	1,179	755	353	2.28
IDDFS	1,003	2,002	383	7	1.11
BFS	10,003	5,307	1,766	3,541	2.13
UCS	10,003	8,318	3,828	3,548	11.58
IDDFS	10,003	115,389	23,076	9	42.03
BFS	100,003	42,447	14,145	28,301	41.73
UCS	100,003	110,472	74,231	27,930	426.0
IDDFS	100,003	1,562,553	319,351	11	641.9
BFS	1,000,003	339,565	113,194	226,370	244.5
UCS	1,000,003	295,934	105,639	159,404	1000.2
IDDFS	1,000,003	DNF	DNF	DNF	>865.2 (stopped)

Table 15: Reachable goal: solution quality (optimal cost per size: 32, 33, 50, 42)

Algorithm	|V|	Depth	Path cost (s)	Optimal cost?
BFS	1,003	6	41	No (fewest moves)
UCS	1,003	7	32	Yes
IDDFS	1,003	6	41	No (fewest moves)
BFS	10,003	8	43	No (fewest moves)
UCS	10,003	9	33	Yes
IDDFS	10,003	8	43	No (fewest moves)

Algorithm	|V|	Depth	Path cost (s)	Optimal cost?
BFS	100,003	10	68	No (fewest moves)
UCS	100,003	15	50	Yes
IDDFS	100,003	10	68	No (fewest moves)
BFS	1,000,003	12	59	No (fewest moves)
UCS	1,000,003	17	42	Yes
IDDFS	1,000,003	-	-	No route returned

Table 16: Unreachable goal: search effort

Algorithm	|V|	Generated	Expanded	Max frontier	Time (ms)
BFS	1,003	1,000	1,000	617	0.729
UCS	1,003	1,228	1,000	353	1.58
IDDFS	1,003	DNF	DNF	DNF	>797.5 (stopped)
BFS	10,003	10,000	10,000	6,219	8.74
UCS	10,003	12,087	10,000	3,659	31.25
IDDFS	10,003	DNF	DNF	DNF	>832.7 (stopped)
BFS	100,003	100,000	100,000	64,876	135.7
UCS	100,003	121,144	100,000	27,930	541.0
IDDFS	100,003	DNF	DNF	DNF	>874.3 (stopped)
BFS	1,000,003	1,000,000	1,000,000	658,667	1592.1
UCS	1,000,003	1,209,244	1,000,000	277,029	8061.7
IDDFS	1,000,003	DNF	DNF	DNF	>755.4 (stopped)

Table 17: Basic-operation counts (successor checks), reachable goal

Algorithm	|V|	Successor checks
BFS	1,003	678
UCS	1,003	3,178
IDDFS	1,003	2,403
BFS	10,003	10,269
UCS	10,003	17,988

Algorithm	|V|	Successor checks
IDDFS	10,003	143,399
BFS	100,003	82,201
UCS	100,003	304,469
IDDFS	100,003	1,921,631
BFS	1,000,003	659,081
UCS	1,000,003	556,912
IDDFS	1,000,003	DNF

For the case start = goal every algorithm generates 1 node, expands 0 nodes, returns depth 0 and cost 0 at all four sizes, in under 0.02 ms. Peak memory of the search structures (tracemalloc, graph excluded) at 1,000,003 states was 16.7 MB for BFS and 37.0 MB for UCS (reachable goal), and 66.7 MB and 152.6 MB for exploring the whole graph.

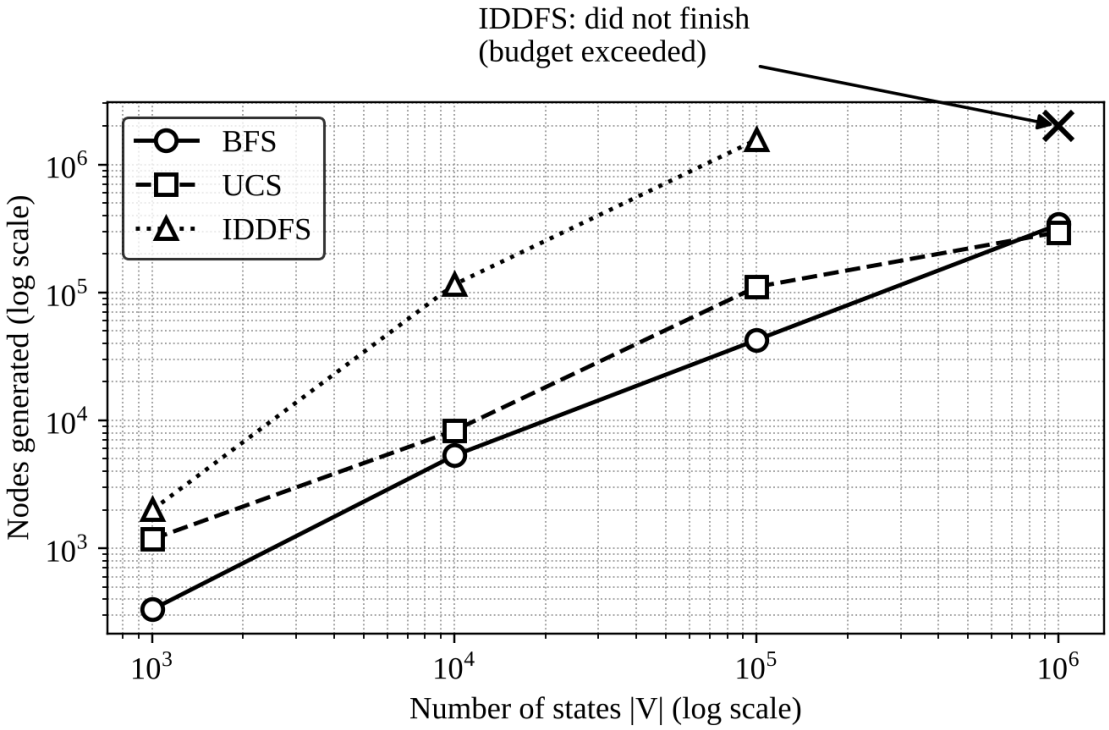


Figure 9: Nodes generated versus number of states (log–log scale, reachable goal)

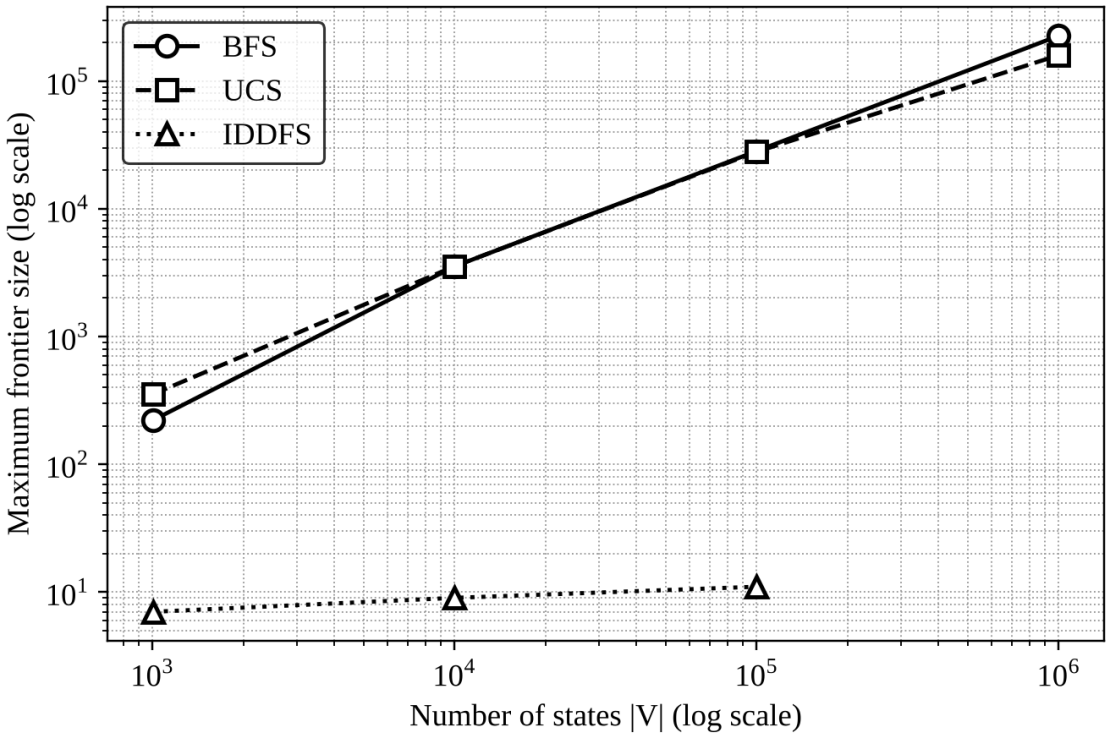


Figure 10: Maximum frontier size versus number of states (log–log scale)

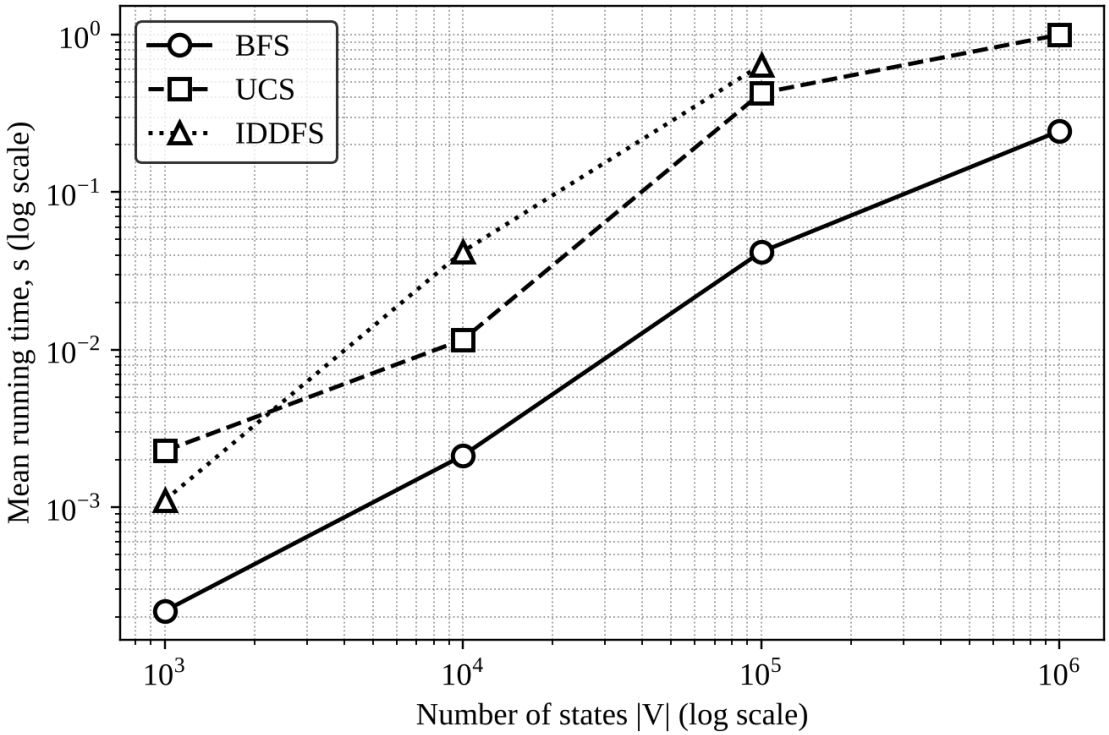


Figure 11: Mean running time versus number of states (log–log scale, 5 runs)

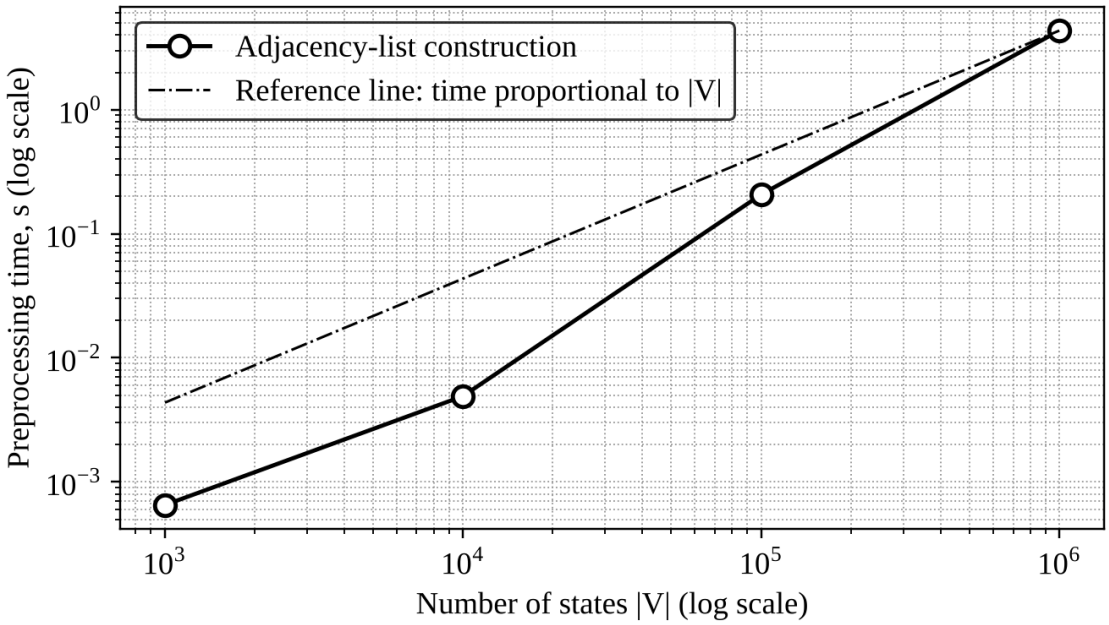


Figure 12: Preprocessing (adjacency-list) time versus number of states

TASK 10 – Compare Theory with Experimental Behaviour

10.1 Nodes generated and frontier size

- **BFS.** Theory predicts $\Theta(\min(b^d, V + E))$ with $b^d \approx V$ in this logarithmic-depth hospital. Observed: generated nodes were 33%, 53%, 42% and 34% of V, a log–log slope of 1.003, i.e. linear growth, and the frontier was 22.6% of V. Theory and experiment agree. The fraction differs from the $(b + 1)/(2b)$ estimate because the real fan-out is irregular and cross-links make many nodes reachable at shallower depth.
- **UCS.** Theory gives $O(V \log V)$ and exploration up to cost C^*. Observed: 1.18 V, 0.83 V, 1.10 V and 0.30 V generated; the last value is low only because the goal at 1,000,003 states had a small C^* (42) compared with the largest costs in the graph. Over 20 random goals UCS used about 0.8 V. The frontier stayed within 0.16–0.35 V.
- **IDDFS.** Theory predicts memory O(d): the observed maximum depth was exactly $d + 1 = 7$, 9 and 11 entries. Time theory $\Theta(b^d)$ predicts a constant factor $b/(b - 1) = 1.5$ over BFS in a tree. Observed IDDFS/BFS ratios were 6.0, 21.7 and 36.8 and rose with size: they exceed the tree prediction because loops give many simple paths to the same state and IDDFS has no explored set. The effective growth per extra two levels (57.6 and 13.5 times) is larger than $b^2 \approx 9$ for the first step.

10.2 Which algorithm scales better

BFS scales best in time and nodes (0.26 s at 1,000,003 states for the reachable goal), UCS costs about 3 times more at 1,000,003 states (0.84 s) and 14 times more at 100,003 states but is the only one that returns the least-time route, and IDDFS scales worst in time but best in memory. Time per generated node was about 0.8 µs for BFS, 2.8 µs for UCS and 0.4 µs for IDDFS at 100,003 states, which shows that UCS pays for the heap (the O(log n) factor of the theory).

10.3 Differences between theory and observation, and reasons

- **Graph structure:** the tree-model formulas assume b^i new nodes per level; in the hospital graph the count saturates at V and cross-links create duplicates, so BFS and UCS follow V + E and IDDFS exceeds b^d.
- **Duplicates:** the explored set removes duplicates for BFS and UCS; IDDFS checks only the current path, so a state reached by k different paths is generated k times.

- **Tie-breaking and goal choice:** the adjacency order decides which nodes of the goal's level are generated first, so BFS counts for one goal vary around the average (33%–53% of V). A single random goal per size also explains the non-monotonic UCS fractions.
- **Language overhead:** time grew by a factor of about 2,550 for a 1,000-fold growth of V (BFS), more than the node ratio of 1,020, because of Python dictionary resizing, memory allocation and cache misses at large sizes. Absolute times are therefore larger than in a compiled language, but the growth in nodes, which is the theoretical quantity, is reliable.
- **Optimality:** BFS and IDDFS return the fewest-move route, which was not the cheapest: BFS cost 41, 43, 68 and 59 against optimal 32, 33, 50 and 42, and 19 of 20 random goals were non-optimal at both 1,003 and 10,003 states (mean cost ratio 1.27 and 1.34, maximum 1.9).

TASK 11 – Recommend and Defend the Final Design

Recommendation: uniform-cost search with adjacency lists, a hash-based explored set and a one-time connected-component labelling, with BFS used only for the preprocessing flood fill.

Table 18: Evidence-based comparison

Criterion	BFS	UCS	IDDFS
Least travel time	No (+27% to +40% cost)	Yes, always	No
Time at 1,000,003 states (reachable)	0.26 s	0.84 s	Did not finish
Time, whole graph explored	1.45 s	7.56 s	Did not finish
Peak memory at 1,000,003 states	16.7 MB (66.7 MB worst)	37.0 MB (152.6 MB worst)	d + 1 entries
Unreachable goal	Terminates	Terminates	Does not terminate in budget
Scales to 10^6 states and beyond	Yes	Yes	No

Why UCS. The performance measure is travel time, and only UCS is optimal for it: BFS routes were 28%, 30%, 36% and 40% longer than optimal in the four tests. UCS is complete and its measured cost is acceptable: 0.84 s for the reachable goal and 7.56 s even when the whole 1,000,003-state map is searched. IDDFS is unsuitable although its memory is minimal: its time grows exponentially with loops and it failed at 1,000,003 states and on every unreachable goal. Choosing the lowest Big-O is not enough: BFS has the lowest time but the wrong objective, and IDDFS has the lowest memory but cannot be used at scale.

Efficiency. Theory: $O((V + E) \log V)$ time and $O(V + E)$ memory. Observation: nodes generated up to 1.1 V and a log–log slope below 1, 36.96 MB peak at the largest size.

Preprocessing and trade-offs. Adjacency-list construction is $\Theta(V + E)$ and took 3.97 s for 1,000,003 states; it is paid once and repeated only when a wing is added (or updated incrementally in O(new edges)). A component-labelling flood fill (about 1.5 s, one BFS) is also

done once, so unreachable requests are answered in O(1) instead of the 7.56 s of exhaustive search. UCS spends more time per node than BFS (heap factor) and uses about twice the memory.

Limitations and growth. UCS explores in all directions because it has no knowledge of where the goal lies. At about 10^7 states the worst-case memory (about 1.5 GB extrapolated) would exceed a typical onboard computer, so beyond that size the design should move to informed search (A* with a distance heuristic) or hierarchical routing, which is outside the uninformed scope of CO1. For 10^3 to 10^6 states the recommended design remains practical.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact analysis

What has changed. After a power restart or sensor fault the robot knows only that it is at one of k possible junctions, yet it must still reach the pharmacy docking station from whichever junction it actually occupies.

- **Known start state** is violated: the initial state is no longer a single node but a set of k candidates.
- **Full observability** is violated: the position can no longer be sensed, so the environment is partially observable (sensorless in the worst case).
- Determinism of the moves still holds, and the map, actions and goal are unchanged.

Why knowledge becomes part of the state. The right action depends on what the robot believes, not only on where it is. The same physical junction needs different actions if the robot believes it is at J2 or at J5, and one command must be safe for every candidate position. The agent therefore searches in the space of **belief states**, where a belief state b is the set of physical states the robot might be in. The initial belief is b_0 = the k candidates, the goal belief is {dock}, and the transition is RESULT(b, a) = { result(s, a) : s in b }. Actions need the same label for every junction (for example corridor exits L, R, U, D), and a blocked move leaves the robot in place.

B. Evaluation of the original design

Can the original algorithm be applied? Not directly. Running BFS or UCS from each of the k candidates gives k different routes, but the robot cannot know which one to follow. A state becomes a set of junctions, the goal test becomes b = {dock}, and the successor function applies the action to every member. The principles of the original design (frontier, explored set, adjacency lists, optimal strategies BFS/UCS) remain valid; the state representation does not.

Size of the new search space. In the worst case any non-empty subset of V can be a belief state, giving $2^n - 1$ states. Because an action never increases the size of a belief, starting from k candidates only subsets of at most k members are reachable:

$$|\text{belief space}| \leq \Sigma_{i=1}^{k} C(n, i) = O(n^k) \text{ for fixed k.}$$

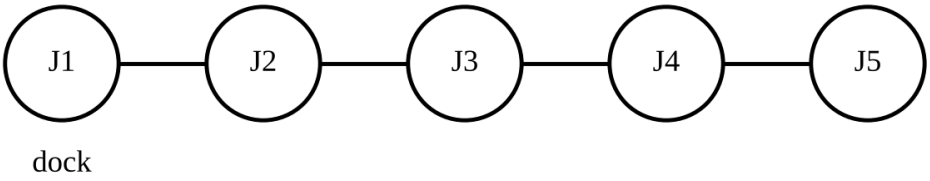
For $n = 10^6$ junctions this is about $n^2/2 = 5 \times 10^{11}$ for k = 2 and about $n^8/8! \approx 2.5 \times 10^{43}$ for k = 8, so searching the full belief space with the original design is **not acceptable** for the large hospital. For the 25-junction floor used in the experiment the bounds are 325, 15,275 and 1,807,780 for k = 2, 4, 8.

C. Adaptation or redesign

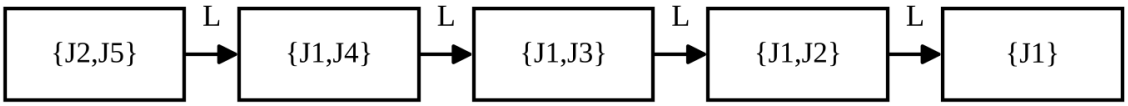
Decision: MODIFY. The original design is not retained unchanged (wrong state) and a full redesign is unnecessary, because the search machinery is reused with a new state definition.

- **Without a local sensor:** sensorless (conformant) search. BFS over belief states finds one action sequence that brings every candidate to the dock; belief states are stored as frozen sets in the explored set. Each expansion costs $O(|b| \cdot |A|)$, so $T = O(N_b \cdot k \cdot |A|)$ with N_b the number of reachable belief states (at most $\Sigma\, C(n, i)$).
- **With a local sensor** (for example the set of open exits at a junction): search with sensing. After each action the predicted belief is split by the percept into smaller beliefs (an AND branch), and the robot follows a **conditional plan** ("if percept p then plan_p"). Beliefs after splitting contain only junctions that look the same, so they are usually very small and the number of belief states stays close to n instead of growing like n^k.
- **Hybrid:** once the belief collapses to one junction the robot switches back to the original UCS plan, so the original algorithm and preprocessing are fully reused.

(a) Corridor: J1 is the docking station; actions L and R



(b) Without sensor: one action sequence for the whole belief



(c) With a local sensor (wall on the left?): the belief splits

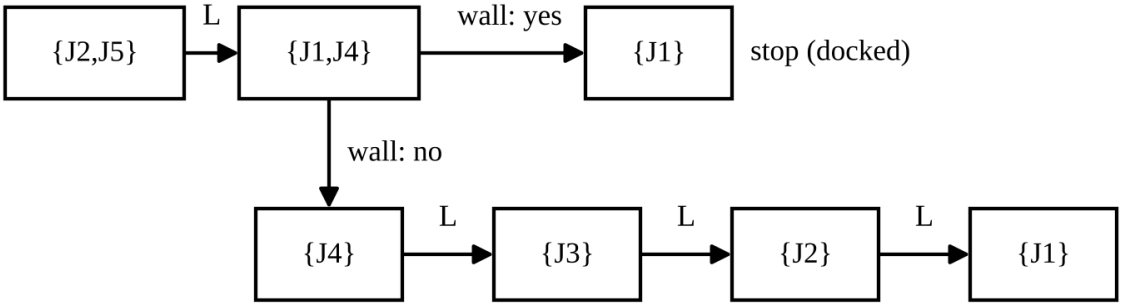


Figure 13: Belief states in a five-junction corridor: (a) corridor, (b) sensorless plan, (c) plan with a wall sensor

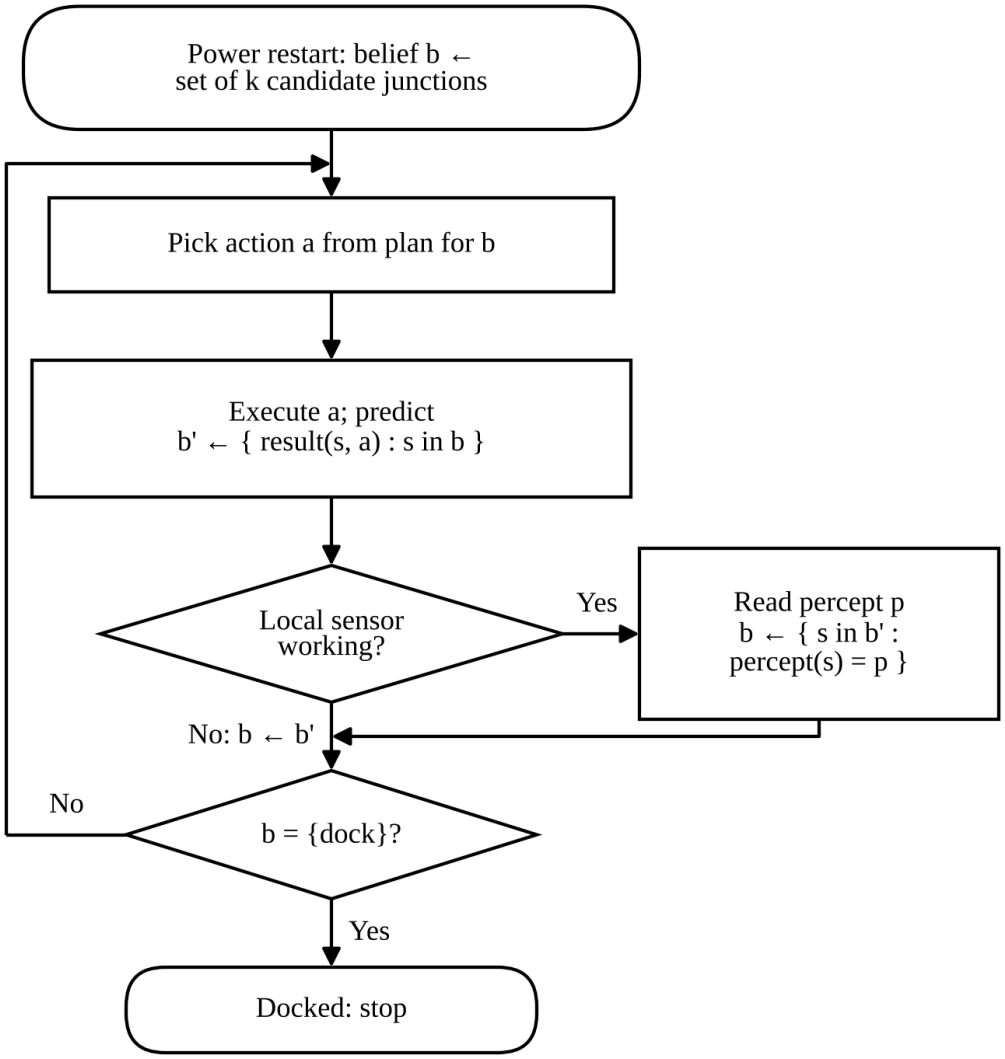


Figure 14: Flowchart of the revised robot behaviour with belief updating (sensor available or not)

Effect on complexity. Sensorless: from $\Theta(V + E)$ to $O(n^k)$ in the worst case but only the reachable subsets are generated (below). With a sensor: the belief-state count is close to n and the plan length approaches the physical worst-case route length, i.e. the complexity is again of the order of the original design times a small factor.

D. Evidence-based defence

Experiment. A 5 × 5 hospital floor (25 junctions, 31 open corridor links, random walls, seed 11) has the dock at junction (0, 0). For each k in {2, 4, 8}, 20 random sets of k candidate junctions (seeds 100 + 10k + t) were solved (a) without a sensor by BFS over belief states and (b) with a local sensor reporting the open exits, by building the percept-split belief graph and computing the minimum worst-case conditional plan length. Code is in Appendix A and the

output in Appendix B. For a robot whose position is known, the worst-case route to the dock is 9 steps.

Table 19: Belief-state experiment (mean over 20 initial sets; maximum in brackets)

k	Upper bound Σ C(25, i)	Belief states, no sensor	Plan length, no sensor	Belief states, sensor	Plan length, sensor
2	325	120.5 (230)	6.1 (9)	25.1 (26)	5.5 (9)
4	15,275	1025.6 (2450)	7.2 (9)	25.4 (27)	6.2 (9)
8	1,807,780	6372.1 (10852)	8.3 (9)	26.9 (29)	7.9 (9)

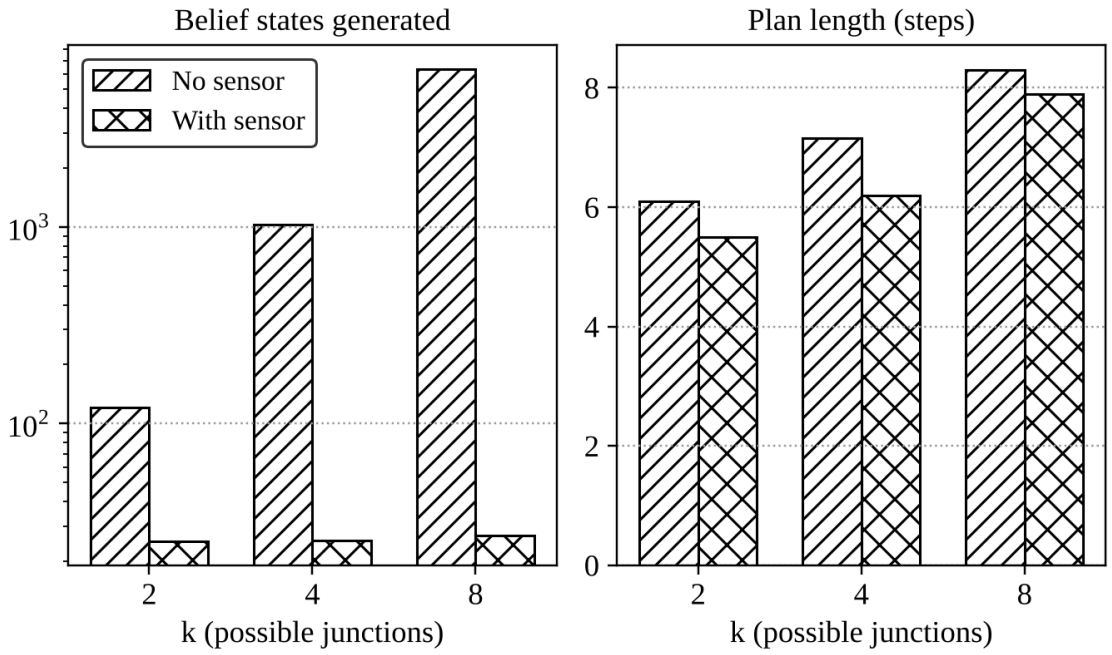


Figure 15: Belief states generated (log scale) and plan length for k = 2, 4, 8, with and without a local sensor

Interpretation. All 60 instances were solved in both modes. Without a sensor the number of belief states generated grows with k (120.5 → 1,025.6 → 6,372.1 on average, a 53-fold increase), though it stays far below the upper bounds (37%, 6.7% and 0.35% of them) because many subsets are not reachable; the plan also lengthens (6.1 → 7.2 → 8.3 steps) since more positions must be brought into agreement. With a sensor the belief count stays at 25.1, 25.4 and 26.9 (about one belief per junction), almost independent of k, and the plans are shorter (5.5, 6.2, 7.9) because a percept splits the belief and lets the robot stop or turn as soon as it is identified. The maximum plan length never exceeds the known-position bound of 9. (For the sensor case all reachable belief states are enumerated, which is a conservative count compared

with BFS stopping at the goal.) The mathematical analysis and the experiment together justify the decision to MODIFY the original design: keep the search engine, change the state to a belief state, and use sensing whenever available.

APPENDIX A – Source Code

A.1 hospital_search.py (graph generator, preprocessing and the three algorithms)

```
"""CSA17 Assignment 1 - Hospital medicine-delivery robot: uninformed search.
Algorithms: BFS (iterative, FIFO queue), UCS (iterative, priority queue),
IDDFS (recursive depth-limited DFS repeated with increasing limits).
"""
import random
import sys
import time
from collections import deque
from heapq import heappush, heappop

sys.setrecursionlimit(10000)



# ---------------------------------------------------------------------
# Hospital graph generator (synthetic, reproducible through a seed)
# ---------------------------------------------------------------------
def generate_edges(n_main, seed, cross_ratio=0.10, island=3):
    """Edge list (u, v, w) of a hospital with n_main connected locations plus a
    small sealed island of `island` locations (used as an unreachable goal).
    Backbone: corridor tree of fan-out about 3 (pharmacy = node 0).
    Cross-links: extra loops between nearby locations on the same level."""
    rng = random.Random(seed)
    edges = []
    for i in range(1, n_main):
        p = max(0, min(i - 1, (i - 1) // 3 + rng.randint(-1, 1)))
        edges.append((p, i, rng.randint(1, 9)))
    seen = {(min(u, v), max(u, v)) for u, v, _ in edges}
    for _ in range(int(cross_ratio * n_main)):
        a = rng.randrange(1, n_main - 6)
        b = a + rng.randint(2, 6)
        if (a, b) not in seen and b < n_main:
```

```
                seen.add((a, b))
                edges.append((a, b, rng.randint(1, 9)))
    base = n_main
    for j in range(island - 1):                    # sealed ward block
        edges.append((base + j, base + j + 1, rng.randint(1, 9)))
    return edges, n_main + island


# ---------------------------------------------------------------------
# Preprocessing: adjacency lists, O(V + E)
# ---------------------------------------------------------------------
def build_adjacency(edges, n):
    adj = [[] for _ in range(n)]
    wt = [[] for _ in range(n)]
    for u, v, w in edges:
        adj[u].append(v); wt[u].append(w)
        adj[v].append(u); wt[v].append(w)
    return adj, wt


# ---------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------
def make_stats():
    return {"generated": 0, "expanded": 0, "max_frontier": 0, "succ_checks": 0}


def rebuild(parent, g):
    path = []
    while g is not None:
        path.append(g)
        g = parent[g]
    return path[::-1]
```

```
def path_cost(path, adj, wt):
    total = 0
    for a, b in zip(path, path[1:]):
        total += wt[a][adj[a].index(b)]
    return total


class BudgetExceeded(Exception):
    pass


# ---------------------------------------------------------------------
# 1. Breadth-first search (iterative, queue, explored set)
# ---------------------------------------------------------------------
def bfs(adj, wt, s, g, budget=None):
    st = make_stats()
    if s == g:
        st["generated"] = 1
        return [s], st
    parent = {s: None}                  # doubles as explored/frontier set
    q = deque([s])
    st["generated"] = 1
    st["max_frontier"] = 1
    while q:
        u = q.popleft()
        st["expanded"] += 1
        for v in adj[u]:
            st["succ_checks"] += 1
            if v not in parent:
                parent[v] = u
                st["generated"] += 1
                if v == g:              # goal test on generation
                    return rebuild(parent, g), st
```

```
                q.append(v)
        if len(q) > st["max_frontier"]:
            st["max_frontier"] = len(q)
    return None, st


# ---------------------------------------------------------------------
# 2. Uniform-cost search (iterative, priority queue, explored set)
# ---------------------------------------------------------------------
def ucs(adj, wt, s, g, budget=None):
    st = make_stats()
    dist = {s: 0}
    parent = {s: None}
    closed = set()
    heap = [(0, s)]
    st["generated"] = 1
    in_frontier = 1
    st["max_frontier"] = 1
    while heap:
        d, u = heappop(heap)
        if u in closed:                  # stale (superseded) entry
            continue
        if u == g:                       # goal test on expansion
            return rebuild(parent, g), st
        closed.add(u)
        in_frontier -= 1
        st["expanded"] += 1
        au, wu = adj[u], wt[u]
        for i in range(len(au)):
            v = au[i]
            st["succ_checks"] += 1
            if v in closed:
                continue
            nd = d + wu[i]
```

```
            old = dist.get(v)
            if old is None or nd < old:
                if old is None:
                    in_frontier += 1
                dist[v] = nd
                parent[v] = u
                heappush(heap, (nd, v))
                st["generated"] += 1
        if in_frontier > st["max_frontier"]:
            st["max_frontier"] = in_frontier
    return None, st


# ---------------------------------------------------------------------
# 3. Iterative-deepening DFS (recursive depth-limited search)
#    Duplicate control: only cycle checking on the CURRENT path (O(m) memory)
# ---------------------------------------------------------------------
def iddfs(adj, wt, s, g, max_limit=None, budget=2_000_000):
    st = make_stats()
    if max_limit is None:
        max_limit = len(adj) - 1          # longest possible simple path
    on_path = set()
    path = []

    def dls(u, limit):
        """returns 'found', 'cutoff' or 'fail'"""
        st["generated"] += 1
        if st["generated"] > budget:
            raise BudgetExceeded
        if u == g:                        # goal test on entry
            path.append(u)
            return "found"
        if limit == 0:
            return "cutoff"
```

```
            st["expanded"] += 1
            on_path.add(u)
            cut = False
            for v in adj[u]:
                st["succ_checks"] += 1
                if v in on_path:                # cycle check on current path
                    continue
                r = dls(v, limit - 1)
                if r == "found":
                    path.append(u)
                    on_path.discard(u)
                    return "found"
                if r == "cutoff":
                    cut = True
            on_path.discard(u)
            return "cutoff" if cut else "fail"

        for limit in range(max_limit + 1):
            on_path.clear()
            r = dls(s, limit)
            st["max_frontier"] = max(st["max_frontier"], limit + 1)
            if r == "found":
                return path[::-1], st
            if r == "fail":
                return None, st
        return None, st


ALGOS = {"BFS": bfs, "UCS": ucs, "IDDFS": iddfs}


def run_algo(name, adj, wt, s, g, runs=5, budget=2_000_000):
    """Run an algorithm `runs` times; returns dict with stats and mean time.
    Returns status 'DNF' if the node budget of IDDFS is exceeded."""
```

```
    fn = ALGOS[name]
    times = []
    res = None
    for _ in range(runs):
        t0 = time.perf_counter()
        try:
            if name == "IDDFS":
                p, st = fn(adj, wt, s, g, budget=budget)
            else:
                p, st = fn(adj, wt, s, g)
        except BudgetExceeded:
            times.append(time.perf_counter() - t0)
            res = {"status": "DNF", "path": None, "stats": None}
            break
        times.append(time.perf_counter() - t0)
        res = {"status": "ok" if p is not None else "nosol", "path": p, "stats": st}
    res["time"] = sum(times) / len(times)
    res["runs"] = len(times)
    return res
```

A.2 run_experiments.py (experiment driver for Tasks 8–10)

```
"""Experiment driver for Tasks 8-10. Seeds: graph = 2026, goal selection = 7."""
import json
import random
import time
from collections import deque

from hospital_search import (generate_edges, build_adjacency, run_algo,
                             path_cost, bfs, ucs, iddfs, ALGOS,
                             BudgetExceeded)

GRAPH_SEED = 2026
GOAL_SEED = 7
CROSS = 1.0
```

```
SIZES = [1000, 10000, 100000, 1000000]
BUDGET = 2_000_000
RUNS = 5


def bfs_depths(adj, s):
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


def valid_path(path, adj, s, g):
    if path is None or path[0] != s or path[-1] != g:
        return False
    return all(b in adj[a] for a, b in zip(path, path[1:]))


def summarise(res, adj, wt, opt_cost):
    if res["status"] != "ok":
        return {"status": res["status"], "time": res["time"]}
    p = res["path"]
    c = path_cost(p, adj, wt)
    out = dict(res["stats"])
    out.update({"status": "ok", "depth": len(p) - 1, "cost": c,
                "optimal": c == opt_cost, "time": res["time"],
                "valid": True})
    return out
```

```
results = {"meta": {"graph_seed": GRAPH_SEED, "goal_seed": GOAL_SEED,
                    "cross_ratio": CROSS, "budget": BUDGET, "runs": RUNS},
           "sizes": {}}

for N in SIZES:
    t0 = time.perf_counter()
    edges, n = generate_edges(N, GRAPH_SEED, cross_ratio=CROSS)
    t_gen = time.perf_counter() - t0
    t0 = time.perf_counter()
    adj, wt = build_adjacency(edges, n)
    t_pre = time.perf_counter() - t0
    E = len(edges)
    depths = bfs_depths(adj, 0)
    rng = random.Random(GOAL_SEED)
    goal = rng.choice([x for x in depths if depths[x] >= 1])
    unreachable = n - 1                    # inside sealed island
    entry = {"V": n, "E": E, "pre_time": t_pre, "gen_time": t_gen,
             "max_depth": max(depths.values()),
             "avg_degree": 2 * E / n, "goal": goal,
             "goal_depth": depths[goal], "scen": {}}
    print(f"N={N}: V={n} E={E} pre={t_pre:.2f}s goal={goal} depth={depths[goal]}",
          flush=True)
    # optimal cost via UCS (reference)
    p_ref, _ = ucs(adj, wt, 0, goal)
    opt = path_cost(p_ref, adj, wt)
    for scen, g in [("reachable", goal), ("unreachable", unreachable),
                    ("start_eq_goal", 0)]:
        entry["scen"][scen] = {}
        for name in ALGOS:
            r = run_algo(name, adj, wt, 0, g, runs=RUNS, budget=BUDGET)
            if scen == "reachable":
                if r["status"] == "ok":
                    assert valid_path(r["path"], adj, 0, g), (name, N)
```

```
                s = summarise(r, adj, wt, opt)
            elif scen == "start_eq_goal":
                s = summarise(r, adj, wt, 0)
            else:
                s = {"status": r["status"], "time": r["time"]}
                if r["stats"]:
                    s.update(r["stats"])
            entry["scen"][scen][name] = s
            print("  ", scen, name, s, flush=True)
    results["sizes"][str(N)] = entry
    del adj, wt, edges, depths

# ---- best / average / worst case study (N = 1000 and 10000) -----------
cases = {}
for N in [1000, 10000]:
    edges, n = generate_edges(N, GRAPH_SEED, cross_ratio=CROSS)
    adj, wt = build_adjacency(edges, n)
    depths = bfs_depths(adj, 0)
    rng = random.Random(GOAL_SEED)
    pool = [x for x in depths if depths[x] >= 1]
    goals = rng.sample(pool, 20)
    best_goal = adj[0][0]                  # first branch explored
    deepest = max(depths, key=lambda x: depths[x])
    case = {"best": {}, "average": {}, "worst_deepest": {}, "worst_unreach": {}}
    for name in ALGOS:
        def one(g):
            try:
                if name == "IDDFS":
                    return iddfs(adj, wt, 0, g, budget=BUDGET)
                return ALGOS[name](adj, wt, 0, g)
            except BudgetExceeded:
                return None, None
        p, st = one(best_goal); case["best"][name] = st
        gens = []; costs = []
```

```
        for g in goals:
            p, st = one(g)
            gens.append(st["generated"] if st else None)
        ok = [x for x in gens if x is not None]
        case["average"][name] = {"mean_generated": sum(ok) / len(ok) if ok else None,
                                 "finished": len(ok), "of": len(goals)}
        p, st = one(deepest); case["worst_deepest"][name] = st
        p, st = one(n - 1); case["worst_unreach"][name] = st
    # BFS optimality gap across the 20 goals
    nonopt = 0; gap = []
    for g in goals:
        pb, _ = bfs(adj, wt, 0, g); pu, _ = ucs(adj, wt, 0, g)
        cb, cu = path_cost(pb, adj, wt), path_cost(pu, adj, wt)
        if cb > cu:
            nonopt += 1
        gap.append(cb / cu)
    case["bfs_nonoptimal"] = nonopt
    case["bfs_mean_ratio"] = sum(gap) / len(gap)
    case["bfs_max_ratio"] = max(gap)
    case["deepest_depth"] = depths[deepest]
    cases[str(N)] = case
    print("cases", N, json.dumps(case)[:400], flush=True)
results["cases"] = cases
json.dump(results, open("results.json", "w"), indent=1)
print("saved")
```

A.3 test_correctness.py (correctness tests)

```
"""Correctness tests (Task 8): tiny hand-made hospital + 300 random small graphs."""
import random
from hospital_search import *

# Tiny hospital: 0=Pharmacy 1=Lift 2=Corridor 3=Ward A 4=Ward B 5=Ward C 6,7=sealed store
E = [(0, 1, 2), (0, 2, 5), (1, 2, 1), (1, 3, 7), (2, 4, 2), (3, 4, 1), (4, 5, 3), (6, 7, 1)]
```

```
adj, wt = build_adjacency(E, 8)
print("TEST 1: tiny hospital  (expected: BFS/IDDFS fewest hops, UCS least time)")
for name, g in [("reachable Ward C", 5), ("start = goal", 0), ("unreachable store", 7)]:
    for a in ALGOS:
        r = run_algo(a, adj, wt, 0, g, runs=1, budget=100000)
        p = r["path"]
        print(f"  {name:18s} {a:5s} path={p} cost={path_cost(p, adj, wt) if p else None}
        status={r['status']}")

def bellman_ford(n, E, s):
    d = [float('inf')] * n; d[s] = 0
    for _ in range(n):
        for u, v, w in E:
            if d[u] + w < d[v]: d[v] = d[u] + w
            if d[v] + w < d[u]: d[u] = d[v] + w
    return d

rng = random.Random(2026)
bad = 0; tested = 0
for t in range(300):
    n = rng.randint(5, 12)
    E2 = []
    for i in range(1, n):
        if rng.random() < 0.9: E2.append((rng.randrange(i), i, rng.randint(1, 9)))
    for _ in range(rng.randint(0, 6)):
        a, b = rng.sample(range(n), 2)
        if not any({a, b} == {x, y} for x, y, _ in E2): E2.append((a, b, rng.randint(1, 9)))
    adj2, wt2 = build_adjacency(E2, n)
    ref = bellman_ford(n, E2, 0)
    hops = {0: 0}; q = [0]
    for u in q:
        for v in adj2[u]:
            if v not in hops: hops[v] = hops[u] + 1; q.append(v)
    for g in range(n):
```

```
        pb, _ = bfs(adj2, wt2, 0, g); pu, _ = ucs(adj2, wt2, 0, g); pi, _ = iddfs(adj2, wt2, 0, g)
        tested += 1
        if g not in hops:
            ok = pb is None and pu is None and pi is None
        else:
            ok = (len(pb) - 1 == hops[g] and len(pi) - 1 == hops[g]
                  and path_cost(pu, adj2, wt2) == ref[g]
                  and all(b in adj2[a] for p in (pb, pu, pi) for a, b in zip(p, p[1:])))
        bad += not ok
print(f"TEST 2: {tested} start/goal pairs on 300 random graphs -> failures: {bad}")
```

A.4 belief_states.py (Task 12 belief-state search)

```
"""Task 12 - localisation failure: search over belief states (k possible junctions).
Floor model: R x C junction grid; actions U, D, L, R are labelled exits, so one
command can be sent to every hypothesised position. A blocked move leaves the
robot where it is. Dock (pharmacy docking station) = cell (0, 0).
  Case A  sensorless : conformant plan found by BFS over belief states.
  Case B  local sensor: sensor reports the set of open exits at the current
                        junction; a conditional plan is computed on the
                        percept-split belief graph (worst-case plan length).
"""
import itertools
import json
import math
import random
from collections import deque

R, C = 5, 5
DOCK = 0
ACTIONS = "UDLR"
DELTA = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}


def build_floor(seed=11, wall_ratio=0.25):
```

```
    """Grid with random internal walls (corridor doors closed), kept connected."""
    rng = random.Random(seed)
    links = set()
    for r in range(R):
        for c in range(C):
            if c + 1 < C: links.add(((r, c), (r, c + 1)))
            if r + 1 < R: links.add(((r, c), (r + 1, c)))
    cand = list(links); rng.shuffle(cand)
    for e in cand[:int(wall_ratio * len(cand))]:
        trial = links - {e}
        if connected(trial):
            links = trial
    return links


def connected(links):
    adj = {}
    for a, b in links:
        adj.setdefault(a, []).append(b); adj.setdefault(b, []).append(a)
    seen = {(0, 0)}; st = [(0, 0)]
    while st:
        u = st.pop()
        for v in adj.get(u, []):
            if v not in seen:
                seen.add(v); st.append(v)
    return len(seen) == R * C


def make_model(links):
    idx = lambda rc: rc[0] * C + rc[1]
    step = {a: [0] * (R * C) for a in ACTIONS}
    percept = [0] * (R * C)
    for r in range(R):
        for c in range(C):
```

```
            s = r * C + c
            for k, a in enumerate(ACTIONS):
                dr, dc = DELTA[a]
                t = (r + dr, c + dc)
                if t[0] in range(R) and t[1] in range(C) and \
                        (((r, c), t) in links or (t, (r, c)) in links):
                    step[a][s] = idx(t)
                    percept[s] |= 1 << k
                else:
                    step[a][s] = s
    return step, percept


def result(step, belief, a):
    return frozenset(step[a][s] for s in belief)


def split(percept, belief):
    groups = {}
    for s in belief:
        groups.setdefault(percept[s], set()).add(s)
    return [frozenset(g) for g in groups.values()]


# ---------------- Case A: sensorless (conformant) BFS ------------------
def sensorless(step, init, cap=300000):
    goal = frozenset([DOCK])
    if init == goal:
        return 0, 1
    seen = {init: None}
    q = deque([(init, 0)])
    gen = 1
    while q:
        b, d = q.popleft()
```

```
        for a in ACTIONS:
            nb = result(step, b, a)
            if nb not in seen:
                seen[nb] = (b, a); gen += 1
                if nb == goal:
                    return d + 1, gen
                if gen > cap:
                    return None, gen
                q.append((nb, d + 1))
    return None, gen                          # no conformant plan


# ---------------- Case B: local sensor, conditional plan ---------------
def with_sensor(step, percept, init):
    goal = frozenset([DOCK])
    roots = split(percept, init)
    succ = {}                                 # belief -> {a: [beliefs]}
    seen = set(roots); q = deque(roots)
    while q:
        b = q.popleft()
        if b == goal:
            continue
        succ[b] = {}
        for a in ACTIONS:
            nxt = split(percept, result(step, b, a))
            if nxt == [b]:
                continue                      # useless self loop
            succ[b][a] = nxt
            for nb in nxt:
                if nb not in seen:
                    seen.add(nb); q.append(nb)
    INF = math.inf
    val = {b: (0 if b == goal else INF) for b in seen}
    changed = True
```

```
    while changed:                          # Bellman iteration
        changed = False
        for b in seen:
            if b == goal:
                continue
            best = min((1 + max(val[x] for x in nxt)
                        for nxt in succ[b].values()), default=INF)
            if best < val[b]:
                val[b] = best; changed = True
    plan_len = max(val[r] for r in roots)
    return (None if plan_len == INF else plan_len), len(seen)


def experiment(trials=20, seed0=100):
    links = build_floor()
    step, percept = make_model(links)
    n = R * C
    out = {"n": n, "k": {}}
    for k in (2, 4, 8):
        rows = []
        for t in range(trials):
            rng = random.Random(seed0 + 10 * k + t)
            init = frozenset(rng.sample(range(1, n), k))
            la, ga = sensorless(step, init)
            lb, gb = with_sensor(step, percept, init)
            rows.append({"init": sorted(init), "plan_A": la, "gen_A": ga,
                         "plan_B": lb, "gen_B": gb})
        ok_a = [r for r in rows if r["plan_A"] is not None]
        ok_b = [r for r in rows if r["plan_B"] is not None]
        mean = lambda xs: sum(xs) / len(xs) if xs else None
        out["k"][str(k)] = {
            "bound": sum(math.comb(n, i) for i in range(1, k + 1)),
            "solved_A": len(ok_a), "solved_B": len(ok_b), "trials": trials,
            "gen_A_mean": mean([r["gen_A"] for r in ok_a]),
```

```
            "gen_A_max": max([r["gen_A"] for r in ok_a], default=None),
            "plan_A_mean": mean([r["plan_A"] for r in ok_a]),
            "plan_A_max": max([r["plan_A"] for r in ok_a], default=None),
            "gen_B_mean": mean([r["gen_B"] for r in ok_b]),
            "gen_B_max": max([r["gen_B"] for r in ok_b], default=None),
            "plan_B_mean": mean([r["plan_B"] for r in ok_b]),
            "plan_B_max": max([r["plan_B"] for r in ok_b], default=None),
            "example": rows[0]}
    # single-state (known position) baseline: worst-case BFS depth to dock
    d = {DOCK: 0}; q = deque([DOCK])
    while q:
        u = q.popleft()
        for a in ACTIONS:
            v = step[a][u]
            if v not in d:
                d[v] = d[u] + 1; q.append(v)
    out["known_position_max_depth"] = max(d.values())
    out["links"] = len(links)
    out["floor"] = [percept[s] for s in range(n)]
    return out, links


if __name__ == "__main__":
    res, links = experiment()
    print("floor links:", res["links"], " percept masks (U,D,L,R bits):")
    for r in range(R):
        print("  ", [res["floor"][r * C + c] for c in range(C)])
    print("known-position worst-case route length:", res["known_position_max_depth"])
    print("k | bound sum C(25,i) | solved A/B | belief gen A (mean,max) | plan A
    (mean,max)"
          " | belief gen B (mean,max) | plan B (mean,max)")
    for k, v in res["k"].items():
        f = lambda x: "-" if x is None else (f"{x:.1f}" if isinstance(x, float) else str(x))
        print(k, v["bound"], f"{v['solved_A']}/{v['solved_B']} of {v['trials']}",
```

```
            (f(v["gen_A_mean"]), f(v["gen_A_max"])), (f(v["plan_A_mean"]),
            f(v["plan_A_max"])),
            (f(v["gen_B_mean"]), f(v["gen_B_max"])), (f(v["plan_B_mean"]),
            f(v["plan_B_max"])))
json.dump(res, open("belief_results.json", "w"), indent=1)
```

APPENDIX B – Program Outputs

B.1 Output of test_correctness.py

```
TEST 1: tiny hospital  (expected: BFS/IDDFS fewest hops, UCS least time)
    reachable Ward C   BFS   path=[0, 2, 4, 5] cost=10 status=ok
    reachable Ward C   UCS   path=[0, 1, 2, 4, 5] cost=8 status=ok
    reachable Ward C   IDDFS path=[0, 2, 4, 5] cost=10 status=ok
    start = goal       BFS   path=[0] cost=0 status=ok
    start = goal       UCS   path=[0] cost=0 status=ok
    start = goal       IDDFS path=[0] cost=0 status=ok
    unreachable store  BFS   path=None cost=None status=nosol
    unreachable store  UCS   path=None cost=None status=nosol
    unreachable store  IDDFS path=None cost=None status=nosol
TEST 2: 2554 start/goal pairs on 300 random graphs -> failures: 0
```

B.2 Output of belief_states.py

```
floor links: 31  percept masks (U,D,L,R bits):
  [10, 12, 14, 14, 4]
  [11, 6, 11, 13, 6]
  [11, 7, 11, 12, 5]
  [11, 15, 15, 6, 2]
  [9, 13, 5, 9, 5]
known-position worst-case route length: 9
k | bound sum C(25,i) | solved A/B | belief gen A (mean,max) | plan A (mean,max) | belief gen B (mean,max) | plan B (mean,max)
2 325 20/20 of 20 ('120.5', '230') ('6.1', '9') ('25.1', '26') ('5.5', '9')
4 15275 20/20 of 20 ('1025.6', '2450') ('7.2', '9') ('25.4', '27') ('6.2', '9')
8 1807780 20/20 of 20 ('6372.1', '10852') ('8.3', '9') ('26.9', '29') ('7.9', '9')
```

B.3 Log of run_experiments.py (key lines)

```
N=1000: V=1003 E=1919 pre=0.00s goal=390 depth=6
reachable BFS generated: 333, expanded: 112, max_frontier: 221, succ_checks: 678, status: ok, depth: 6, cost: 41, optimal: False, time: 0.00022, valid: True
reachable UCS generated: 1179, expanded: 755, max_frontier: 353, succ_checks: 3178, status: ok, depth: 7, cost: 32, optimal: True, time: 0.00228, valid: True
```

reachable IDDFS generated: 2002, expanded: 383, max_frontier: 7, succ_checks: 2403, status: ok, depth: 6, cost: 41, optimal: False, time: 0.00111, valid: True
unreachable BFS status: nosol, time: 0.00073, generated: 1000, expanded: 1000, max_frontier: 617, succ_checks: 3834
unreachable UCS status: nosol, time: 0.00158, generated: 1228, expanded: 1000, max_frontier: 353, succ_checks: 3834
unreachable IDDFS status: DNF, time: 0.79752
N=10000: V=10003 E=19043 pre=0.00s goal=6355 depth=8
reachable BFS generated: 5307, expanded: 1766, max_frontier: 3541, succ_checks: 10269, status: ok, depth: 8, cost: 43, optimal: False, time: 0.00213, valid: True
reachable UCS generated: 8318, expanded: 3828, max_frontier: 3548, succ_checks: 17988, status: ok, depth: 9, cost: 33, optimal: True, time: 0.01158, valid: True
reachable IDDFS generated: 115389, expanded: 23076, max_frontier: 9, succ_checks: 143399, status: ok, depth: 8, cost: 43, optimal: False, time: 0.04203, valid: True
unreachable BFS status: nosol, time: 0.00874, generated: 10000, expanded: 10000, max_frontier: 6219, succ_checks: 38082
unreachable UCS status: nosol, time: 0.03125, generated: 12087, expanded: 10000, max_frontier: 3659, succ_checks: 38082
unreachable IDDFS status: DNF, time: 0.83269
N=100000: V=100003 E=190788 pre=0.21s goal=46631 depth=10
reachable BFS generated: 42447, expanded: 14145, max_frontier: 28301, succ_checks: 82201, status: ok, depth: 10, cost: 68, optimal: False, time: 0.04173, valid: True
reachable UCS generated: 110472, expanded: 74231, max_frontier: 27930, succ_checks: 304469, status: ok, depth: 15, cost: 50, optimal: True, time: 0.42600, valid: True
reachable IDDFS generated: 1562553, expanded: 319351, max_frontier: 11, succ_checks: 1921631, status: ok, depth: 10, cost: 68, optimal: False, time: 0.64192, valid: True
unreachable BFS status: nosol, time: 0.13566, generated: 100000, expanded: 100000, max_frontier: 64876, succ_checks: 381572
unreachable UCS status: nosol, time: 0.54100, generated: 121144, expanded: 100000, max_frontier: 27930, succ_checks: 381572
unreachable IDDFS status: DNF, time: 0.87429
N=1000000: V=1000003 E=1906360 pre=4.34s goal=329752 depth=12
reachable BFS generated: 339565, expanded: 113194, max_frontier: 226370, succ_checks: 659081, status: ok, depth: 12, cost: 59, optimal: False, time: 0.24445, valid: True

reachable UCS generated: 295934, expanded: 105639, max_frontier: 159404, succ_checks: 556912, status: ok, depth: 17, cost: 42, optimal: True, time: 1.00021, valid: True
reachable IDDFS status: DNF, time: 0.86524
unreachable BFS status: nosol, time: 1.59214, generated: 1000000, expanded: 1000000, max_frontier: 658667, succ_checks: 3812716
unreachable UCS status: nosol, time: 8.06168, generated: 1209244, expanded: 1000000, max_frontier: 277029, succ_checks: 3812716
unreachable IDDFS status: DNF, time: 0.75537